

HyperMR: Efficient Hypergraph-enhanced Matrix Storage on Compute-in-Memory Architecture

YIFAN WU and KE CHEN, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China

GANG CHEN and DAWEI JIANG, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China

HUAN LI* and LIDAN SHOU*, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China

Matrix-vector multiplication (MVM) operations, essential for modern hardware architectures, suffer from heavy I/O overheads and costly serial multiply-add operations. The emerging Compute-in-Memory (CIM) architecture alleviates these issues by enabling in situ MVM operations with $O(1)$ time complexity, eliminating the need to move matrices. However, current storage schemes are still inefficient on CIM due to limited optimization objectives and inflexible support for various access patterns and matrix structures. To address this, we propose HyperMR, a hypergraph-enhanced matrix storage scheme for CIM architectures. First, we identify two performance optimization objectives that are tailored to CIM and prove their NP-hardness. We then introduce a hypergraph modeling approach with a novel access-aware hypergraph generation algorithm to handle diverse matrix structures and access patterns. Moreover, we present a two-phase hypergraph partitioning method to efficiently tackle the NP-hard optimization objectives. Experimental results show that HyperMR outperforms multiple state-of-the-art storage schemes, offering valid optimization for all evaluated matrices, compared to the best-performing baseline which optimizes only 75%. HyperMR also achieves the best average optimization performance for matrix storage layouts, significantly improving efficiency in varied workload scenarios, with a 29.65% improvement on synthetic queries and up to 34.9% on scientific image filtering.

CCS Concepts: • **Information systems** → **Data layout**; • **Hardware** → **Emerging technologies**.

Additional Key Words and Phrases: Compute-in-Memory Architecture, Matrix Storage, Scientific Data Management, Emerging Hardware

ACM Reference Format:

Yifan Wu, Ke Chen, Gang Chen, Dawei Jiang, Huan Li, and Lidan Shou. 2025. HyperMR: Efficient Hypergraph-enhanced Matrix Storage on Compute-in-Memory Architecture. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 45 (February 2025), 27 pages. <https://doi.org/10.1145/3709695>

*Lidan Shou and Huan Li are the corresponding authors.

Authors' Contact Information: Yifan Wu, yifan.wu@zju.edu.cn; Ke Chen, chenk@zju.edu.cn, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China; Gang Chen, cg@zju.edu.cn; Dawei Jiang, davidjiang2005@gmail.com, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China; Huan Li, lihuan.cs@zju.edu.cn; Lidan Shou, should@zju.edu.cn, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, China and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/2-ART45
<https://doi.org/10.1145/3709695>

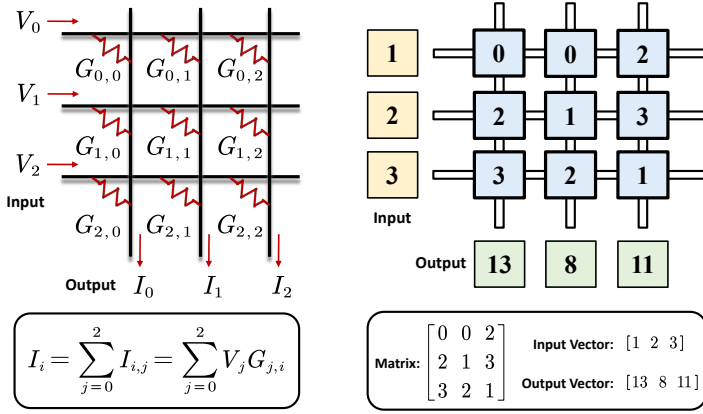


Fig. 1. Perform MVM operations using the CIM architecture. An analog voltage vector (V_0, V_1, V_2) is applied to the rows of the non-volatile crossbar where the conductance of each non-volatile memory cell is given by $G_{0,0} \dots G_{2,2}$. The output current vector (I_0, I_1, I_2) from the columns represents the MVM result based on Ohm's law and Kirchhoff's law.

1 Introduction

Deep learning and scientific computing applications rely heavily on efficient **matrix-vector multiplication (MVM)** operations, which are crucial for processing analytical queries involving matrix data. Popular data systems opt to natively support diverse MVM operations and associated tools to accommodate these critical application scenarios, such as PostgreSQL [24] and SciDB [9]. These systems widely adopt advanced solutions such as Apache MADlib [24] and NVIDIA cuSparse [44] to accelerate MVM-based workloads on CPU and GPU platforms. However, MVM operations typically require loading matrices from disk to memory and then to cache, resulting in significant I/O overhead [7, 18, 21]. To address this, researchers have proposed efficient matrix storage techniques to reduce disk I/Os [27, 31, 63] and operate directly on compressed matrices [7, 18, 21]. Nevertheless, the heavy I/O and computational complexity of MVM operations (essentially as serial multiply-add operations) remain insurmountable.

In this study, we advocate the emerging **Compute-in-Memory (CIM) architecture** [14, 52, 57], which offers the following attractive features: 1) CIM performs MVM operations in $O(1)$ time complexity and eliminates the need to move matrices, significantly reducing I/O overhead. As per a study [59], CIM offers high performance with ultra-low energy usage, achieving up to 106× TOPS per Watt compared to TPU; 2) CIM stores matrices persistently, eliminating the need for external storage, unlike modern accelerators such as GPU and TPU. Recently, CIM has shown immense potential in various MVM-intensive applications such as deep learning [1, 52, 59] and scientific computing [19, 20].

Figure 1 depicts an MVM operation conducted in a non-volatile crossbar. Non-volatile crossbars, the core component of CIM, are non-volatile memory cells (e.g., Flash, ReRAM, and PCM) arranged in grid-like structures. These structures facilitate a reduction in computational complexity, transforming the $O(MN)$ operations of multiplications and additions associated with an $M \times N$ matrix into a constant-time complexity of $O(1)$.

The generic CIM device uses a hierarchical structure of *tiles*, *processing elements* (PEs), and *crossbars* to store large-scale matrices. As shown in Figure 3, each tile contains multiple PEs, and each PE contains multiple crossbars. A matrix, virtually represented as a 2D array, is sliced into fixed-size sub-matrices, each physically stored in one tile (detailed in §2.1). Notably, this tile-based

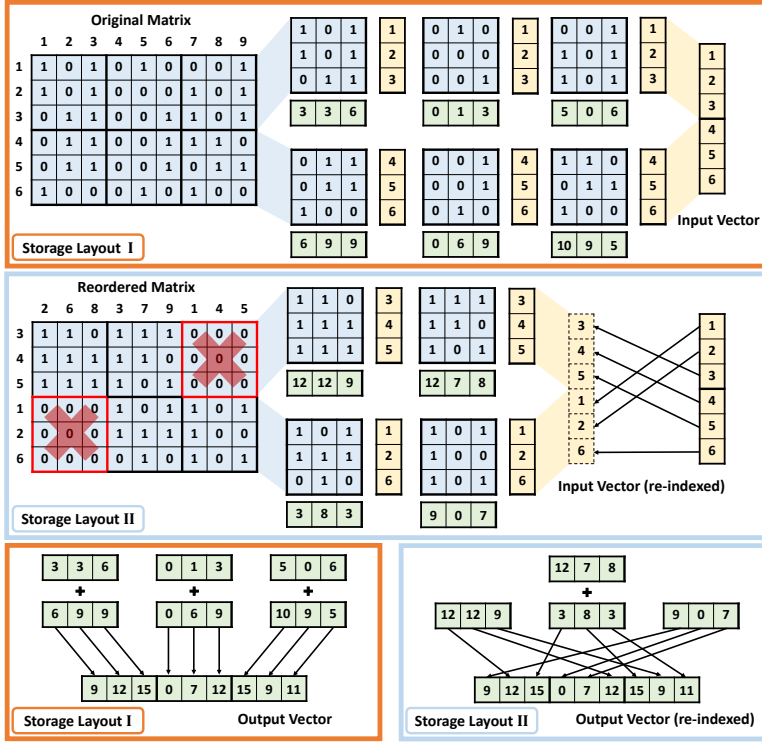


Fig. 2. MVM operations performed using (I) original matrix vs (II) reordered matrix. Red crosses mark empty tiles.

architecture offers significant optimization opportunities: The matrix sparsity (presence of zero elements), commonly seen in real workloads [17], usually leads to wasted memory cells in tiles. Previous works [15, 58] have employed **matrix reordering** techniques (reordering of rows and columns) to move zero elements into specific ‘empty’ tiles. With proper reordering, CIM devices avoid allocating physical tiles for these empty tiles, reducing unnecessary computation.

Figure 2 illustrates the effects of matrix reordering. For simplicity, we use a 3×3 crossbar as one tile, ignoring finer structures. Storage Layout I (orange box) shows the original matrix sliced into six tiles. After reordering the rows as {3,4,5,1,2,6} and columns as {2,6,8,3,7,9,1,4,5}, Storage Layout II (blue box) shows the reordered matrix occupying only four tiles (two empty tiles are freed up). This reordering reduces the dominant costs of query workloads on CIM, specifically the communication cost of assigning the input vector to tiles and the result accumulation cost (see our analysis in §3).

Inspired by the benefits of matrix reordering, various storage schemes have been proposed [2, 15, 20, 25, 35, 40, 58]. However, these schemes have significant limitations:

L1 (Inadequate Optimization Objectives). Previous methods focused on rearranging zero matrix elements to maximize idle tiles. However, our cost analysis (§3) shows this approach is insufficient for improving MVM efficiency. The primary bottleneck is communication cost in CIM. Column order determines communication costs, while the number of used tiles impacts accumulation costs. This calls for new approaches that optimize performance objectives.

L2 (Dependency on Specific Matrix Structures). Previous methods assume specific structural patterns in matrices. For example, GSMR [15] requires a diagonal block pattern; METIS [20], based on graph partitioning, supports only symmetric matrices; GMR [40] assumes non-zero elements

are aggregated in one matrix corner; ReSpar [25] assumes that leading zeros of the columns are aggregated in one matrix corner. These assumptions limit their general applicability (see detailed discussions in §7).

L3 (Lack of Flexible Access Pattern Support). Previous methods are designed for typical MVM operations where the input vector is multiplied by the entire matrix. Real-world scenarios, indeed, require support for analytical workloads accessing *arbitrary sub-matrices*. For example, SciDB [9] supports a scientific image filtering workload, Gaussian Smoothing, which performs MVM on stridden sub-matrices of the matrix.

To overcome these limitations, we propose HyperMR, an efficient matrix storage scheme for the CIM architecture. The core idea of HyperMR is to leverage hypergraph modeling to support various matrix structures and access patterns, which is the most significant distinction from existing CIM-based methods. Addressing **L1**, we first perform a detailed cost analysis of CIM workload processing, identifying two key optimization objectives: communication-optimal column order and compression-optimal row order. For **L2**, we introduce hypergraph models to represent matrices, allowing adaptation to diverse matrix structures. Further addressing **L3**, we devise an access-aware hypergraph generation algorithm that integrates storage layout and access patterns into a unified hypergraph, enabling HyperMR to handle different access patterns.

Despite hypergraphs supporting various matrix structures and access patterns, we prove that addressing the two aforementioned optimization objectives is NP-hard (see Appendix §A.3 [5]). To tackle this, we develop a novel two-phase hypergraph partitioning approach that transforms optimizing these objectives into hypergraph partitioning problems, leveraging advanced *balanced k-way hypergraph partitioning* techniques [49]. We compare HyperMR with six state-of-the-art CIM-based storage schemes across matrices of varied structures and workloads of diverse access patterns. HyperMR offers valid optimization¹ for all evaluated matrices, while the best baseline optimizes only 75%. It also achieves the highest average optimization performance for storage layouts, improving efficiency by 29.65% on synthetic queries and up to 34.9% on scientific image filtering. Our key contributions are as follows:

- We propose HyperMR, a matrix storage scheme specifically designed for the emerging CIM hardware. To our best knowledge, HyperMR is the first CIM-based storage scheme that supports various matrix structures and access patterns (see §2).
- We present cost modeling and performance analysis to scrutinize MVM processing on CIM, identifying two key optimization objectives for effective matrix reordering (see §3).
- We propose a hypergraph modeling approach and a novel access-aware hypergraph generation algorithm to handle various matrix structures and access patterns in matrix reordering (see §4 & 5.1); We also propose a two-phase hypergraph partitioning approach to effectively tackle the NP-hard optimization objectives within hypergraph modeling (see §5.2).
- Extensive experiments across various baselines, matrices, and workloads confirm that our method surpasses existing storage schemes in execution time while achieving compressed layouts, effectively optimizing complex matrices (see §6).

In addition, we review the related work in §7 and conclude the paper in §8.

2 Preliminaries

In this section, we first introduce the generic CIM architecture, prototype hardware, and two primitive operations (§2.1). Then, we elaborate on the framework of HyperMR, the specific workloads it targets, and the delineation of the research problem (§2.2).

¹As per §6.1, a storage scheme is considered to provide valid optimization if it meets the following criteria: 1) the compression ratio of the optimized matrix is ≥ 1 ; 2) the execution time after optimization is lower than that of the unoptimized matrix.

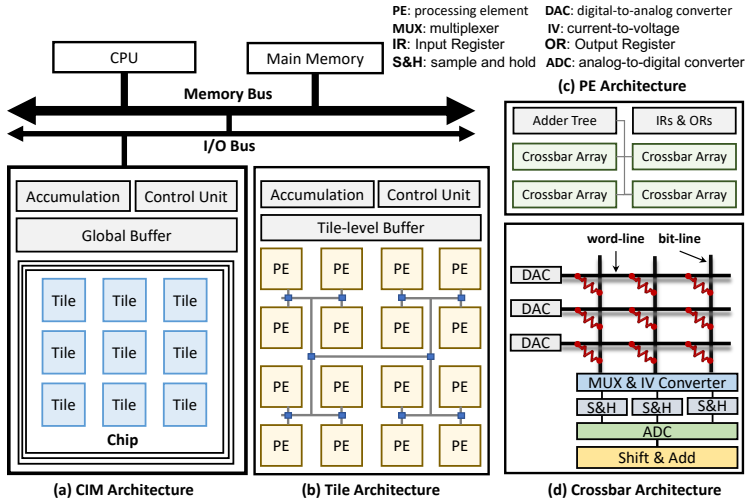


Fig. 3. The Compute-in-Memory architecture.

2.1 Compute-in-Memory (CIM)

Architecture. The CIM architecture is a promising architecture among the various Processing-in-Memory (PIM) architectures [51]. In this work, we introduce the widely-used ReRAM-based CIM architecture [14, 52, 57]. This CIM device is usually attached to the host via an I/O bus. It typically employs a hierarchical architecture to accommodate large-scale matrices, as depicted in Figure 3. The generic CIM architecture (see Figure 3(a)) consists of multiple tiles connected by interconnects, a global buffer, an accumulation unit, and a control unit [37]. The control unit is mainly responsible for storing device status, I/O control, and instruction execution. The accumulation unit is mainly responsible for accumulating the output results of all tiles. The global buffer is composed of a data buffer and an instruction buffer. The data buffer consists of an input buffer and an output buffer for caching the input and output data. Similarly, each tile (see Figure 3(b)) consists of multiple processing elements (PEs), a tile-level buffer, and a tile-level accumulation unit, as well as a control unit. Each PE (see Figure 3(c)) consists of an adder tree, crossbar arrays, input registers (IRs), and output registers (ORs). The IR and OR are used to store input operands and output results, respectively. Each crossbar (see Figure 3(d)) is equipped with peripheral circuits to implement in situ MVM operations, including multiplexers (MUXs), sample-and-hold (S&H) circuits, shift-and-add (S&A) units, digital-to-analog converters (DACs), analog-to-digital converters (ADCs), and a current-to-voltage (IV) converter.

Prototype Hardware. The non-volatile crossbar structure is the core of CIM. Flash, PCM, and ReRAM can be used as the crossbar's memory cell. We list the representative prototype hardware for each non-volatile CIM as follows. (1) Flash-based CIM: The Mythic M1076 Analog Matrix Processor [22, 42] is a commercial chip that delivers up to 25 TOPS and 18.5 TOPS/W (TOPS per Watt). Its commercial product can store 304 MB of data [43]. (2) PCM-based CIM: IBM HERMES Chip [33] can deliver up to 63.1 TOPS and 9.7 TOPS/W. It can store 4 MB of data. (3) ReRAM-based CIM: TSMC has demonstrated a 4 MB ReRAM-based CIM chip can deliver up to 24.8 TOPS and 251 TOPS/W [26]. These prototype hardware, based on the hierarchical architecture depicted in Figure 3, can scale storage capacity up to the gigabyte level [52, 57]. ReRAM-based CIM hardware is considered the most promising and commercially viable solution due to its superior energy efficiency and manufacturability. Leading manufacturers like TSMC have already demonstrated the capability

to mass-produce this hardware [1]. Therefore, our work is based on a generic ReRAM-based CIM architecture.

Write Operation. The matrix is written to the CIM device via a write operation initiated from the host. The matrix data is transferred from the main memory to the CIM device's global buffer via the I/O bus. After transmission, the matrix is stored through the following steps: (1) The matrix is sliced into sub-matrices based on a fixed tile size, with each tile holding one sub-matrix. These sub-matrices are transferred to their respective tile and stored in the tile-level buffer; (2) Each sub-matrix in the tile-level buffer is further sliced by the fixed PE size and transferred to the corresponding PE where input registers (IRs) store the received data; (3) The IR contents holding the sub-matrices for that PE are then written *in parallel by row* to each crossbar.

MVM Operation. Given a stored matrix and an input vector in the global buffer, an MVM operation proceeds in the following three phases. (1) *Communication* Phase: the input vector is read from the global buffer and distributed to each tile-level buffer; (2) *Tile Execution* Phase: the input vector is read from the tile-level buffer and distributed to the PEs for computation. The results from all PEs are accumulated and written back to the tile-level buffer. This phase completes when the last executing tile finishes its job; (3) *Accumulation* Phase: computation results from each tile are accumulated and written back to the global buffer. We refer interested readers to a comprehensive survey [12] for further descriptions of the procedures inside the PE. The core advantage of the CIM architecture is its parallel execution of tiles and built-in accumulators, which reduce the overhead associated with tile execution and result accumulation. Despite CIM's focused optimizations on the Tile Execution and Accumulation phases, the Communication phase still incurs significant performance overhead. Our performance analysis in §3 identifies communication costs as a critical bottleneck.

2.2 HyperMR on CIM

Query Workloads and Access Patterns. First, we introduce the two types of query workloads that CIM devices can handle²:

- Single-MVM query: Given a matrix A , a query may need to perform a single MVM operation on A or its sub-matrix.
- Multi-MVM query: Given a matrix A , a query may need to perform multiple MVM operations on A and/or its sub-matrices.

Example 1. Given a matrix A of size 6×9 ,

$Q1: O_1 = \text{MVM}(A[1 : 6][1 : 9], V_1)$, where V_1 is a 1×6 input vector;

$Q2: O_2 = \text{MVM}(A[1 : 4][1 : 4], V_2)$, and

$O_3 = \text{MVM}(A[1 : 4][3 : 6], V_2)$, where V_2 is a 1×4 input vector.

Example 1 shows two queries. Q_1 is a single-MVM query which incurs an MVM operation O_1 on matrix A of size 6×9 with an input vector V_1 . Q_2 is a multi-MVM query which incurs two MVM operations O_2 and O_3 on two different sub-matrices of A . Specifically, O_2 performs MVM on a sub-matrix of A (rows 1–4 and columns 1–4) and input vector V_2 , while O_3 performs MVM on another sub-matrix of A (rows 1–4 and columns 3–6) and V_2 . Query Q_2 essentially performs a convolution computation on A employing a 4×4 kernel with a stride size of 2. Note that single-MVM queries are special cases of multi-MVM queries. Otherwise specified, we focus on processing multi-MVM queries in this study.

²CIM is optimized for MVM-based workloads, ensuring exceptional performance. While not primarily designed for other data processing types, such as relational data, its specialized architecture excels in its intended applications.

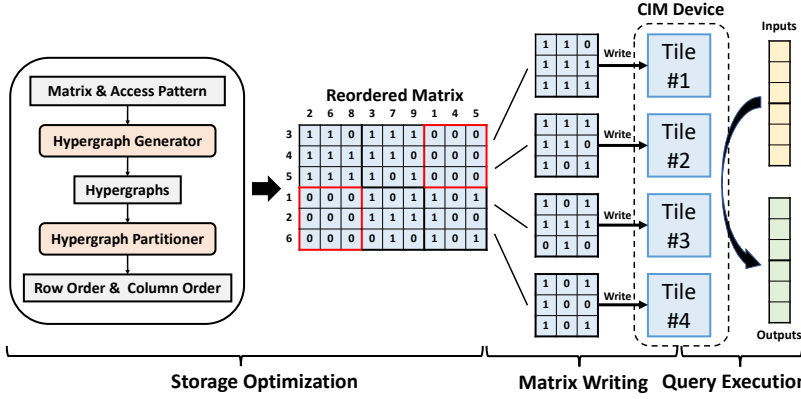


Fig. 4. The HyperMR framework.

To model the access characteristics of multi-MVM queries, we introduce the concept of *Access Pattern* as follows.

Definition 1 (Access Pattern). Given a matrix A of size $M \times N$ and a set Q of multi-MVM queries, the access pattern of Q is represented as the probability distribution over all unique sub-matrices accessed by Q . Assume there are K unique sub-matrices, with the i -th sub-matrix denoted by R_i . The probability of Q accessing R_i , denoted $\mathcal{P}(R_i)$, equals the fraction of queries in Q that access R_i . A query is said to access R_i if and only if it utilizes all rows and columns of R_i for computation. Consequently, the access pattern w.r.t Q is the set of probabilities corresponding to all K unique sub-matrices:

$$\mathbb{P} = \{\mathcal{P}(R_1), \mathcal{P}(R_2), \dots, \mathcal{P}(R_K)\} \quad (1)$$

We present Example 2 to illustrate how access patterns can be mapped and calculated from the real-world application.

Example 2 (Access Patterns in Image Filtering). Consider a common scientific image filtering workload in array databases. Suppose that a 49×49 convolution kernel is applied to an image with a stride of 32. Suppose a matrix A of size 1024×1024 represents this image. Given the convolution kernel, it requires 961 convolutions on the image. Each convolution incurs a multi-MVM query consisting of 49 MVM operations because a convolution kernel can be regarded as 49 distinct input vectors. Thus, the entire workload generates a set of 961 multi-MVM queries, denoted as Q . Each multi-MVM query q_i accesses a 49×49 region of the image, denoted as R_i , and q_i accesses R_i 49 times. Consequently, the access pattern of Q is $\mathbb{P} = \{\mathcal{P}(R_1), \mathcal{P}(R_2), \dots, \mathcal{P}(R_{961})\} = \{0.001, 0.001, \dots, 0.001\}$.

Leveraging access patterns can optimize query performance and hardware resource utilization. Traditional storage schemes assume a uniform access probability across the entire matrix, which is not suitable for workloads like scientific image filtering that require frequent access to specific sub-matrices (i.e., convolution filtering of local regions of the image) [9]. These schemes scatter frequently accessed sub-matrices across more tiles, increasing communication and accumulation costs. Therefore, optimizing matrix reordering based on access patterns is crucial. However, modeling both matrix storage layouts and access patterns is challenging. To resolve this, our scheme HyperMR involves advanced hypergraph models for matrix storage optimization, as to be detailed in §4.2 & 5.1.

HyperMR Overview. As illustrated in Figure 4, the HyperMR framework consists of three processing phases:

- **Storage Optimization.** The fundamental technique of HyperMR is storage optimization. HyperMR takes the given access pattern and the matrix to be stored, to produce a reordered matrix storage layout. The access pattern can be derived from a set of user-specified queries or historical query statistics (see Definition 1). We develop two modules for storage optimization, namely a *hypergraph generator* (§5.1) and a *hypergraph partitioner* (§5.2). The hypergraph generator takes as input the access pattern and the matrix to produce intermediate hypergraphs. Then, the hypergraph partitioner partitions hypergraphs to obtain a desirable row and column order. Our partitioner aims to provide performance gains on the given access pattern without allocating more tiles. The hypergraph modeling and associated techniques take into account the characteristics of access patterns and are not limited to specific matrix structures.
- **Matrix Writing.** HyperMR takes the reordered matrix, performs matrix tiling, and writes the tiled matrix to the CIM device.
- **Query Execution.** When a query is executed, the input vector is transferred to the corresponding tiles based on the reordered row and column indexes. HyperMR performs in situ MVM operations on the CIM device.

HyperMR targets various practical **MVM-intensive scenarios** with persistent, invariant matrices, such as archived matrix data analytics [19, 20], deep learning inference [52], and scientific image analytics [34, 60]. HyperMR benefits these applications by significantly accelerating the execution time of multi-MVM queries and improving matrix compression ratios.

The overhead of HyperMR is the time spent on *storage optimization* and *matrix writing*, which occur only once during the cold start. The query execution phase in Figure 4 is a typical warm execution phase. Since the query execution phase involves repeated processing of multiple MVM queries, the overhead of the first two phases can be easily amortized in multiple query executions. Moreover, the cold-start overhead of HyperMR is much lower than that of most CIM-based methods.

Our experiments evaluate HyperMR and CIM-based storage schemes regarding both the query execution (§6.2 and 6.4) and cold start phases (§6.3), providing a comprehensive and organized comparison. However, since the matrix writing and query execution are straightforward with CIM, we shall focus on the **storage optimization** in the following presentation of our techniques.

Problem Statement. Existing works [2, 15, 20, 25, 35, 40, 58] focus solely on accelerating single-MVM queries as they do not support arbitrary sub-matrices. In contrast, HyperMR aims to support more generalized multi-MVM queries, which are common in real-world workloads. Specifically, HyperMR seeks to *design an efficient matrix storage layout that accelerates the given multi-MVM queries while also enabling matrix data compression on the CIM device*.

To address this, we proceed to scrutinize the performance of processing MVM operations in CIM in §3. The outcomes enable us to reformulate this problem as an optimization problem in §4.

3 Performance Analysis

In this section, we analyze the CIM architecture's performance bottleneck through performance breakdown and cost modeling, identifying two key optimization objectives for HyperMR.

Performance Breakdown Analysis. As depicted in Figure 5, we conduct a detailed performance analysis of MVM operations using 20 widely-used benchmark datasets, which are sourced from the SuiteSparse Matrix Collection [17] that capture various real workloads (see Table 1). The horizontal axis represents the matrix identifiers, while the left vertical axis indicates the percentage of performance overhead. This overhead comprises the three phases of the MVM operation, as described in §2.1: communication cost, accumulation cost, and tile execution cost. The right vertical axis denotes the size of the matrices. Our experimental results indicate that communication costs significantly hinder performance, with this bottleneck becoming more pronounced for larger

matrices (see detailed measurements in Appendix §A.2[5]). Next, we examine the factors affecting the three cost components.

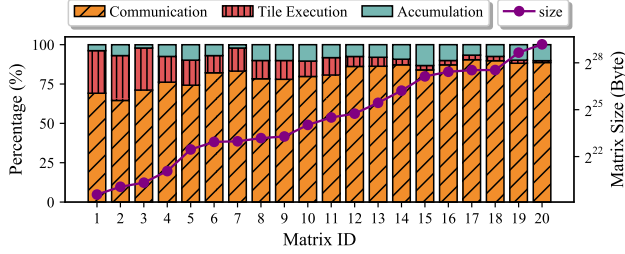


Fig. 5. Time breakdown of performing MVM operations on matrices of different sizes (data from Table 1, §6.1).

Analytical Cost Modeling. Existing works have focused on reordering the matrix to maximize idle tiles, but it is unclear if the resulting row and column orders improve MVM processing efficiency. To address this, we present an analytical cost model for modeling the performance of *Query Execution* phase on a generic and hierarchical CIM architecture (see §2.1). The matrix is pre-stored on tiles, and both input and output data are stored in the global buffer. As introduced for the MVM operation in §2.1, the time cost of processing the MVM operation on CIM is modeled as $T_{\text{overall}} = T_{\text{comm}} + T_{\text{accum}} + T_{\text{tile}}$. We proceed to examine each of the three components.

(1) The communication cost T_{comm} includes two parts: reading the input data from the global buffer and transferring them to the tile-level buffers, and transferring the accumulated output data back to the global buffer. Equation 2 estimates T_{comm} :

$$T_{\text{comm}} = (x_{\text{in}} + x_{\text{out}}) / B_0 \quad (2)$$

Here, B_0 denotes the global buffer bandwidth (GB/sec), x_{in} is the total size of the input data distributing to all used tiles, and x_{out} is the size of the output data. Given that the matrix width determines x_{out} , optimizing x_{in} is crucial. For an MVM operation on an $M \times N$ matrix $\mathbf{A}$ with a dense input vector V , each row i of $\mathbf{A}$ needs to be multiplied by the element $V(i)$ of V . If the non-zero elements of row i are distributed across s_i tiles, $V(i)$ must be communicated to these tiles. The variable s_i indicates the number of CIM tiles needed to store the non-zero elements in the i -th row of matrix $\mathbf{A}$. Thus, x_{in} for this MVM operation is $\sum_{i=1}^M s_i$. This data communication depends on the matrix's column order, which determines the placement of non-zero entries within rows across tiles. Different row orders *do not* affect $\sum_{i=1}^M s_i$, i.e., do not affect the communication cost of the input vector to all CIM tiles, as the variable s_i depends on the matrix's column order. Moreover, x_{out} depends on the matrix width, and thus different row orders do not result in different communication times. Since the arrangement of non-zero elements in each row depends on the column order, finding an optimal column order that minimizes x_{in} is essential to reduce T_{comm} . Therefore, we derive the first optimization objective (**O1**) for HyperMR as follows.

O1. Finding the optimal column order to minimize T_{comm} .

(2) The accumulation cost T_{accum} is the time to gather output results from all used tiles. Equation 3 estimates T_{accum} :

$$T_{\text{accum}} = \sum_{c=1}^C \sum_{i=1}^{\lceil \log_2(N_{\text{tile}}(c)) \rceil} \frac{N_{\text{col}}(c)}{N_{\text{Adder}}} \cdot (k_0 + i \cdot k_1) \quad (3)$$

where C is the number of accumulation groups³, N_{Adder} is the number of adders, $N_{\text{tile}}(c)$ is the number of tiles in the c -th accumulation group, $N_{\text{col}}(c)$ is the number of non-null column indexes in the c -th accumulation group, k_0 is the read latency of the adder, and k_1 is the additional accumulation factor. Both k_0 and k_1 are constants.

After resolving **O1** to minimize communication costs by determining an optimal column order, the next step is to find the optimal row order based on this column order. As shown in Equation 3, T_{accum} depends on the number of used tiles, $\sum_{c=1}^C N_{\text{tile}}(c)$. Minimizing the number of used tiles maximizes the compression ratio. Therefore, with a fixed column order, an optimal row order can be found to minimize the number of tiles used, reducing T_{accum} and improving compression ratios. This defines the second optimization objective (**O2**) for HyperMR:

O2. *With the given column order, finding the optimal row order to reduce the number of tiles used.*

(3) The execution cost T_{tile} is the time to execute all tiles, with parallel execution tracked by the last tile to finish⁴. T_{tile} is a minor performance factor in our analysis. As shown in the measurements in Appendix §A.2 [5], its proportion of total latency decreases from 6.32% to 1.17% as matrix size increases from 0.1 GB to 2.27 GB. Thus, HyperMR does not prioritize optimizing T_{tile} .

In summary, we propose two optimization objectives to accelerate multi-MVM queries and achieve a compressed storage layout: **O1** reduces communication costs to overcome performance bottlenecks, while **O2** reduces accumulation costs and improves compression ratios. We formalize **O1** and **O2** using Definitions 2 and 3, respectively, in §4.1.

4 Problem Modeling via Hypergraph

In this section, we first provide the formal definitions of the two optimization objectives in §4.1. Then, we lay the theoretical foundation for HyperMR by explaining how matrices are modeled as hypergraphs in §4.2.

4.1 Formulation of Optimization Objectives

The row (column) order refers to the sequence of indexes for each row (column) of the matrix. Our analysis regarding **O1** indicates that the column order determines the communication cost (§3). We now refer to this column order as the *Communication-optimal Column Order* and give a formal definition in Definition 2.

Definition 2 (Communication-optimal Column Order). *Given a matrix $\mathbf{A}$ of size $M \times N$ and a set $\mathbf{Q}$ of multi-MVM queries. Each MVM operation in $\mathbf{Q}$ refers to a dense input vector multiplied by a sub-matrix in $\mathbf{A}$. Let $\mathbf{A}'$ be the matrix obtained by permuting the columns of $\mathbf{A}$ according to the communication-optimal column order. The communication cost of $\mathbf{Q}$ executed on $\mathbf{A}'$ is minimized.*

In our analysis about **O2**, given a communication-optimal column order, we can find a good row order to reduce the number of used tiles and accumulation cost. We refer to this row order as the *Compression-optimal Row Order*, formally defined in Definition 3.

Definition 3 (Compression-optimal Row Order). *Given a matrix $\mathbf{A}$ of size $M \times N$ and a column order I_c , let $\mathbf{A}'$ be the matrix obtained by permuting the rows and columns of $\mathbf{A}$ according to the compression-optimal row order and I_c . This row order minimizes the number of used tiles to store the matrix $\mathbf{A}'$ on the CIM device.*

³Tiles with the same column indexes of sub-matrices form an accumulation group.

⁴Detailed modeling of T_{tile} is provided in Appendix §A.1 [5].

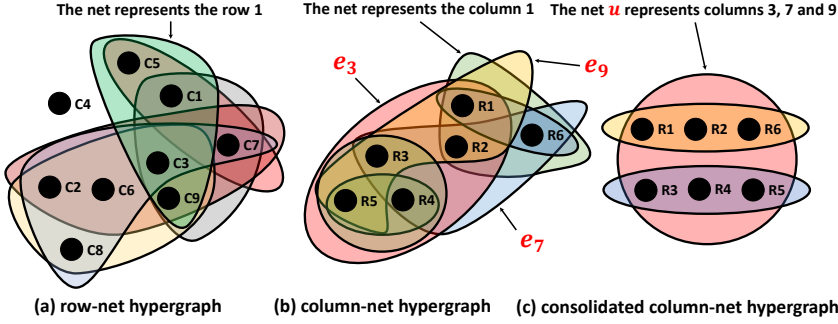


Fig. 6. The hypergraphs that represent the matrices in Figure 2. C1 denotes column 1 and R1 denotes row 1.

4.2 Hypergraph Models for Matrices

Matrix structures can be complex, such as symmetric or asymmetric. Hence, matrices are often modeled as hypergraphs for tasks like matrix partitioning [11] and parallel MVM [10]. Hypergraphs can naturally represent arbitrary matrix structures, overcoming the limitation of prior works that rely on specific matrix structures.

There are two ways to transform matrices into hypergraphs, and we formalize them as *row-net* and *column-net*. We define the hypergraph in Definition 4, and then define the row-net hypergraph and column-net hypergraph in Definitions 5 and 6, respectively.

Definition 4 (Hypergraph). A weighted undirected hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{N}, c, \omega)$ is defined as a set $\mathcal{V}$ of n vertices and a set $\mathcal{N}$ of m hyperedges/nets with vertex weights $c : \mathcal{V} \rightarrow \mathbb{R}^+$ and net weights $\omega : \mathcal{N} \rightarrow \mathbb{R}^+$, where each net e is a subset of the vertex set $\mathcal{V}$ (i.e., $e \subseteq \mathcal{V}$). We denote $c(v)$ and $\omega(e)$ as the weights of v and e , respectively. We extend c and ω to sets in the natural way, i.e., $c(\mathcal{V}) := \sum_{v \in \mathcal{V}} c(v)$ and $\omega(\mathcal{N}) := \sum_{e \in \mathcal{N}} \omega(e)$. A vertex v is incident to a net e if $v \in e$. We assume nets to be sets rather than multisets, i.e., a vertex can only be contained in a net once.

Definition 5 (Row-Net Hypergraph). In the row-net model, a hypergraph $\mathcal{H}_R = (\mathcal{V}_R, \mathcal{N}_R, c, \omega)$ represents the matrix A . Vertex and net sets $\mathcal{V}_R$ and $\mathcal{N}_R$ correspond to the columns and rows of matrix A , respectively. There exists one vertex v_i and one net e_j for each column i and row j , respectively. Net $e_j \subseteq \mathcal{N}_R$ contains the vertices corresponding to the columns that have a non-zero entry in row j . That is, $v_i \in e_j$ if and only if $a_{ji} \neq 0$, where a_{ji} is the entry of the j -th row and i -th column in matrix A .

Definition 6 (Column-Net Hypergraph). In the column-net model, a hypergraph $\mathcal{H}_C = (\mathcal{V}_C, \mathcal{N}_C, c, \omega)$ represents the matrix A . Vertex and net sets $\mathcal{V}_C$ and $\mathcal{N}_C$ correspond to the rows and columns of matrix A , respectively. There exists one vertex v_i and one net e_j for each row i and column j , respectively. Net $e_j \subseteq \mathcal{N}_C$ contains the vertices corresponding to the rows that have a non-zero entry in column j . That is, $v_i \in e_j$ if and only if $a_{ij} \neq 0$, where a_{ij} is the entry of the i -th row and j -th column in matrix A .

Figures 6(a) and (b) show the row-net hypergraph and column-net hypergraph of the matrix in Figure 2. In Figure 6(a), the green region denotes a row-net, which corresponds to the first row of the matrix in Figure 2. Each vertex in the net, namely C1, C3, C5, and C9, represents a column connected to row 1. Similarly, Figure 6(b) indicates that rows R1, R2, and R6 are connected to column 1.

In summary, we have formalized the optimization objectives and introduced hypergraph models to represent matrix storage layouts. Next, we will address these objectives using novel hypergraph methods in §5, based on the theoretical modeling discussed above.

Algorithm 1: Access-aware Hypergraph Generation

Input : a matrix $\mathbf{A}$ of size $M \times N$, a set $\mathcal{Q}$ of multi-MVM queries

Output: an access-aware hypergraph $\mathcal{H}_Q$

```

1  convert  $\mathbf{A}$  to a row-net hypergraph  $\mathcal{H}_R = (\mathcal{V}_C, \mathcal{N}_R, c, \omega)$ ;
2  model  $\mathcal{Q}$  as the access pattern  $\mathbb{P}$ ; /* Definition 1 */
3   $N_Q \leftarrow$  the number of multi-MVM queries in  $\mathcal{Q}$ ;
4   $K \leftarrow$  the number of unique sub-matrices in  $\mathbb{P}$ ;
5  an access-aware hypergraph  $\mathcal{H}_Q = (\mathcal{V}_Q, \mathcal{N}_Q, c, \omega) \leftarrow \emptyset$ ;
6   $\mathcal{V}_Q \leftarrow \mathcal{V}_C$ ;
7  for each  $i \in [1, K]$  do
8      for each  $r \in [H_{start}^{(i)}, H_{end}^{(i)}]$  do
9           $e_r \in \mathcal{N}_R$  and the row  $r$  of  $\mathbf{A}$  corresponds to the net  $e_r$ ;
10         a new net  $e_q \leftarrow \emptyset$ ;
11         for each  $v \in e_r$  do
12             if  $v \in [W_{start}^{(i)}, W_{end}^{(i)}]$  then
13                 add  $v$  to  $e_q$ ;
14             end
15         end
16          $\omega(e_q) \leftarrow N_Q \cdot \mathcal{P}(R_i)$ ;
17         add  $e_q$  to  $\mathcal{N}_Q$ ;
18     end
19 end
  
```

5 Storage Optimization in HyperMR

In this section, we introduce the core techniques of HyperMR to optimize storage layouts. First, we present an access-aware hypergraph generation method that considers both matrix layouts and access patterns in construction (see §5.1). Next, we propose a two-phase hypergraph partitioning method to achieve optimal communication column order and compression-optimal row order, addressing the two optimization objectives (see §5.2). Lastly, we provide a complexity analysis in §5.3.

5.1 Access-aware Hypergraph Generation

To solve the optimization objective **O1**, HyperMR uses a novel access-aware hypergraph generation approach that leverages access patterns to construct a row-net hypergraph. This row-net hypergraph could be partitioned using the method outlined in Section 5.2 to derive the communication-optimal column order.

HyperMR can reorder the matrix using the access pattern of multi-MVM queries, which can be user-specified or derived from historical query statistics. If no access pattern is provided, the default access pattern is to set the access probability to 1 for all values in the matrix. We give the definition of Access Pattern in Definition 1. With the given access pattern, we model the matrix as an access-aware row-net hypergraph from it.

Algorithm 1 shows the procedure of access-aware hypergraph generation. It takes a matrix $\mathbf{A}$ and a set $\mathcal{Q}$ of multi-MVM queries as input and outputs an access-aware hypergraph $\mathcal{H}_Q$. The algorithm first converts $\mathbf{A}$ to a row-net hypergraph $\mathcal{H}_R$ and models $\mathcal{Q}$ as the access pattern $\mathbb{P}$. It then creates an access-aware hypergraph $\mathcal{H}_Q$ by iterating over each sub-matrix in $\mathbb{P}$ and each row in $\mathbf{A}$ that corresponds to the net e_r in $\mathcal{H}_R$. The i -th sub-matrix denoted by $R_i = \{[H_{start}^{(i)}, H_{end}^{(i)}], [W_{start}^{(i)}, W_{end}^{(i)}]\}$, where $H_{start}^{(i)}$ and $H_{end}^{(i)}$ are the indices of the starting and ending rows of R_i , $W_{start}^{(i)}$ and $W_{end}^{(i)}$ are those for the columns of R_i , $1 \leq H_{start}^{(i)} \leq H_{end}^{(i)} \leq M$, and $1 \leq W_{start}^{(i)} \leq W_{end}^{(i)} \leq N$. K denotes the number of unique sub-matrices in $\mathbb{P}$, and $i \in [1, K]$. For each row r , it creates a new net e_q and adds the columns in the same sub-matrix to e_q as vertices. Finally, it sets the weight of each net e_q and adds e_q to $\mathcal{H}_Q$.

Algorithm 2: Two-Phase Hypergraph Partitioning**Input** : a matrix A of size $M \times N$, a set Q of multi-MVM queries, the width of a tile W_{tile} , the height of a tile H_{tile} **Output**: a row order I_r , a column order I_c

```

1 use the query-aware hypergraph generation algorithm to generate  $\mathcal{H}_Q = (\mathcal{V}_Q, \mathcal{N}_Q, c, \omega)$  by  $A$  and  $Q$ ; /* Algorithm 1 */
2  $S_c \leftarrow$  all columns not used by  $Q$ ;
3 convert  $A$  to a row-net hypergraph  $\mathcal{H}_R = (\mathcal{V}_R, \mathcal{N}_R, c, \omega)$ ;
4 for each net  $e \in \mathcal{N}_R$  do
5     create a new net  $e_r \leftarrow \emptyset$ ;
6     for each vertex  $v \in e$  do
7         if  $v \in S_c$  then
8             add  $v$  to  $e_r$ ;
9         end
10    end
11    add  $e_r$  to  $\mathcal{H}_Q$ ;
12 end
13  $k \leftarrow \lceil \frac{c(\mathcal{V}_Q)}{W_{tile}} \rceil$ , partition  $\mathcal{H}_Q$  with KaHyPar to obtain column partitions  $\Pi_c$ ;
14 convert  $A$  to a column-net hypergraph  $\mathcal{H}_C = (\mathcal{V}_C, \mathcal{N}_C, c, \omega)$ ;
15 create a new hypergraph  $\mathcal{H}_M = (\mathcal{V}_M, \mathcal{N}_M, c, \omega) \leftarrow \emptyset$ ;
16 for each partition  $\mathcal{V}_i \in \Pi_c$  do
17     create a new net  $e_c \leftarrow \emptyset$ ;
18     for each column  $j \in \mathcal{V}_i$  do
19         consolidate the net  $e_j$  and the net  $e_c$ ; /* Definition 7 */
20     end
21     add  $e_c$  to  $\mathcal{H}_M$ ;
22 end
23  $k \leftarrow \lceil \frac{c(\mathcal{V}_M)}{H_{tile}} \rceil$ , partition  $\mathcal{H}_M$  with KaHyPar to obtain row partitions  $\Pi_r$ ;
24  $I_r \leftarrow \Pi_r, I_c \leftarrow \Pi_c$ ;

```

5.2 Two-Phase Hypergraph Partitioning

We have conducted a thorough investigation and concluded that resolving the two optimization objectives, **O1** and **O2**, is NP-hard. We reached this conclusion by using hypergraph models introduced in our study. These models transform the problems related to **O1** and **O2** into instances of NP-hard hypergraph partitioning problems. This transformation allowed us to prove their NP-hard complexity. The detailed proofs can be found in Appendix §A.3[5].

To effectively solve **O1** and **O2**, we propose a two-phase hypergraph partitioning approach based on advanced hypergraph partitioning techniques. Its core idea: In the first phase, the matrix and the access pattern are modeled as an access-aware hypergraph, and then the hypergraph is partitioned to find the communication-optimal column order (**O1**); In the second phase, the matrix is modeled as a column-net hypergraph, and then the consolidated column-net hypergraph is generated by the *net consolidation* strategy with the given column order, and finally the hypergraph is partitioned to obtain the compression-optimal row order (**O2**). Each phase of hypergraph partitioning uses the celebrated balanced k -way hypergraph partitioning tool KaHyPar [49]. KaHyPar can provide fast and high-quality solutions for balanced k -way hypergraph partitioning problems.

HyperMR first optimizes the column order independently and then optimizes the row order based on the determined column order. This column-first-row-then policy addresses the main performance bottleneck: communication cost, which is only affected by the column order. Optimizing the column order first is crucial because objective **O2** requires it to be clear before determining the row order (explained in §3). Moreover, HyperMR's policy is less complex than simultaneously optimizing both column and row orders. Additionally, our experimental evaluation shows that this policy can effectively accelerate multi-MVM queries (see §6). Next, we present the details of the two-phase hypergraph partitioning, as shown in Algorithm 2.

Algorithm 3: Row Reordering-Based Compression

Input : a matrix $\mathbf{A}$ of size $M \times N$, a column order I_c , the width of a tile W_{tile} , the height of a tile H_{tile}
Output: a row order I_r

```

1  convert  $\mathbf{A}$  to a column-net hypergraph  $\mathcal{H}_C = (\mathcal{V}_C, \mathcal{N}_C, c, \omega)$ ;
2  create a new hypergraph  $\mathcal{H}_M = (\mathcal{V}_M, \mathcal{N}_M, c, \omega) \leftarrow \emptyset$ ;
3  for each  $i \in [1, N]$  do
4       $e_i \in \mathcal{N}_C$  and the column  $i$  of  $\mathbf{A}$  corresponds to the net  $e_i$ ;
5      create a new net  $e_c \leftarrow \emptyset$ ;
6      for each  $j \in [i, i + W_{tile} - 1]$  do
7          consolidate the net  $e_j$  and the net  $e_c$ ; /* Definition 7 */
8      end
9      add  $e_c$  to  $\mathcal{H}_M$ ;
10      $i \leftarrow i + W_{tile}$ ;
11 end
12  $k \leftarrow \lceil \frac{c(\mathcal{V}_M)}{H_{tile}} \rceil$ , partition  $\mathcal{H}_M$  with KaHyPar to obtain row partitions  $\Pi_r$ ;
13  $I_r \leftarrow \Pi_r$ ;
  
```

In the first phase, the algorithm begins by generating an access-aware hypergraph $\mathcal{H}_Q$ and identifies all columns not used by Q as $\mathcal{S}_c$. The matrix $\mathbf{A}$ is then converted to a row-net hypergraph $\mathcal{H}_R$, and for each net e in $\mathcal{H}_R$, a new net e_r is created consisting of all vertices in e that are also in $\mathcal{S}_c$. These new nets are added to $\mathcal{H}_Q$. Next, the algorithm partitions $\mathcal{H}_Q$ using balanced k -way hypergraph partitioning to obtain the column partitions Π_c .

In the second phase, the matrix $\mathbf{A}$ is then converted to a column-net hypergraph $\mathcal{H}_C$. For each partition $\mathcal{V}_i$ in Π_c , all nets e_j ($j \in \mathcal{V}_i$) are consolidated into a new net e_c by using a *net consolidation* strategy, as defined in Definition 7.

Definition 7 (Net Consolidation). Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{N}, c, \omega)$, and a set of nets $\mathcal{N}'$ are selected from $\mathcal{N}$. The net consolidation adds a new net u into $\mathcal{H}$, where u is the union of all nets in $\mathcal{N}'$, $u = \bigcup_{e \in \mathcal{N}'} e$, and then removes all nets of $\mathcal{N}'$ from $\mathcal{H}$.

Figure 6(c) illustrates the consolidated column-net hypergraph of the reordered matrix in Figure 2. In Figure 6(b), e_3, e_7, e_9 denote the nets of columns 3, 7, and 9, respectively, which are eventually consolidated into the net u in Figure 6(c).

These new nets are added to the consolidated column-net hypergraph $\mathcal{H}_M$. Then, $\mathcal{H}_M$ is partitioned using balanced k -way hypergraph partitioning to obtain the row partitions Π_r . The output is Π_r as I_r and Π_c as I_c . Finally, the vertices within the same row (column) partition in Π_r (Π_c) are arranged together to get I_r (I_c). I_c represents the solution to optimization objective **O1**, and I_r represents the solution to optimization objective **O2**.

The Case of Over-allocated Tiles. The second-phase hypergraph partitioning can lead to the over-allocation of tiles. We propose a compression technique to avoid this. Given the matrix's original column order, our method aims to create a storage layout that uses fewer tiles and reduces accumulation costs. As shown in Algorithm 3, the input is a matrix $\mathbf{A}$ of size $M \times N$, a column order I_c , and tile dimensions W_{tile} and H_{tile} . The output is a row order I_r . The algorithm converts $\mathbf{A}$ to a column-net hypergraph $\mathcal{H}_C$, then creates a new hypergraph $\mathcal{H}_M$. Next, for each $i \in [1, N]$, it consolidates the net $e_i \in \mathcal{N}_C$ corresponding to the i -th column of $\mathbf{A}$ with a new net e_c up to the $(i + W_{tile} - 1)$ -th column, adding e_c to $\mathcal{H}_M$. Finally, $\mathcal{H}_M$ is partitioned into k row partitions Π_r using a balanced k -way hypergraph partitioning algorithm, assigning Π_r to I_r .

5.3 Complexity Analysis

We analyze the complexity of the proposed algorithms. For a matrix $\mathbf{A}$ of size $M \times N$ and a given access pattern $\mathbb{P}$ with K unique sub-matrices, the time complexity of Algorithm 2 in the worst

case is $O(K \cdot M \cdot N + M^2 \cdot N + 2 \cdot t)$. Its first step incurs a complexity of $O(K \cdot M \cdot N)$ in the worst case. At the second step, the row-net/column-net hypergraph generation requires $O(M \cdot N)$, and the net generation step itself demands $O(M \cdot N)$ in the worst case. At the third step, the net consolidation necessitates $O(M^2 \cdot N)$ in the worst case. The final step of Algorithm 2 calls balanced k -way hypergraph partitioning twice, where t is the runtime of balanced k -way hypergraph partitioning, as discussed elsewhere [48]. The time complexity of Algorithm 3 in the worst case is $O(M \cdot N + M^2 \cdot N + t)$ as it only requires the second-phase hypergraph partitioning.

We have measured the time overhead of the hypergraph solution in §6.3, confirming that HyperMR's time overhead is superior to most existing CIM-based solutions. HyperMR optimizes matrix storage layouts during cold start, accelerating multi-MVM queries in the warm execution phase (i.e., query execution phase in §2.2). Executed once during a cold start, its performance gains in warm execution offset its overhead. In this sense, the complexity of the hypergraph solution being harder than that of MVM operations is reasonable in practice.

6 Experimental Studies

This section aims to empirically verify that HyperMR provides high-quality storage layouts in terms of execution time and compression ratio. Our evaluation addresses the following research questions:

- RQ1 Efficiency of MVM:** How does HyperMR's efficiency in processing MVM operations compare to various methods based on CIM, CPU, or GPU?
- RQ2 Time and Storage Overheads:** How do HyperMR's time and storage overheads compare to other CIM-based schemes?
- RQ3 Varied Access Patterns:** How well does HyperMR perform in handling single-MVM and multi-MVM queries under varying access patterns?
- RQ4 Cost Model Validation:** How accurate is the proposed analytical cost model in §3?

6.1 Experimental Setup

Hardware Configuration. To ensure fair comparisons, we use an Intel CPU server to perform storage optimization techniques of all evaluated storage schemes, producing reordered matrices. These matrices are then loaded into the NeuroSim benchmark CIM simulator [13, 39] to evaluate matrix writing and query execution. The server has an Xeon® Gold 6240C CPU @ 2.60GHz, featuring 72 cores, 376 GB of DDR4 DRAM, a 4.4 TB HDD, and a 26 MB cache. NeuroSim is configured with a 1 MB global buffer and includes 256 tiles, each containing 4×4 PEs. Each PE has 8 crossbars, each sized at 128×128 with 2 bits per cell. Each crossbar's read (write) latency is 18.26 ns (93.09 ns). The bandwidth between the CIM device and the main memory is 32 GB/sec. The simulated CIM hardware has a total memory size of 128 MB spread across 256 physical tiles. CIM devices can efficiently store larger matrices in limited memory by avoiding the allocation of physical tiles for all-zero sub-matrices during the matrix writing phase. Given the fine granularity of CIM tiles, this saving is significant. In our experiments, the largest matrix "crystm03" in Table 1 actually occupied 239 physical tiles.

We use NeuroSim to simulate the CIM platform for the following reasons: (1) Commercial CIM hardware is not readily accessible. To the best of our knowledge and inquiries, the Mythic M1076 [42] is the only mass-produced non-volatile CIM chip, but it is not publicly accessible. This Mythic CIM hardware is only provided to lead customers, such as Lockheed Martin [41]. (2) The open-source CIM simulation platform, NeuroSim [13], is designed for proof-of-concept and has been adopted by industry researchers from Intel, Samsung, TSMC, and SK Hynix [39]. It shows a relative error of less than 1% compared to the actual TSMC-produced 40-nm CIM chip [39],

Table 1. Matrices widely used for evaluations [15, 25, 46].

ID	Name	Height	Width	Sparsity	Symmetry	Used Tiles	Size	Application
1	qh882	882	882	0.9957	0.926	4	2.97 MB	Power Network
2	radfr1	1,048	1,048	0.9879	0.0537	7	4.19 MB	Chemical Process Simulation
3	illc1850	1,850	712	0.9934	0	8	5.02 MB	Least Squares Problem
4	qh1484*	1,484	1,484	0.9972	1	9	8.40 MB	Power Network
5	rdist3a	2,398	2,398	0.9892	0.1404	16	21.94 MB	Chemical Process Simulation
6	extr1	2,837	2,837	0.9986	0.0042	21	30.70 MB	Chemical Process Simulation
7	zenios*	2,873	2,873	0.9998	1	10	31.49 MB	Optimization Problem
8	bayer09	3,083	3,083	0.9988	0.0212	29	36.26 MB	Chemical Process Simulation
9	rdist2	3,198	3,198	0.9944	0.0456	21	39.01 MB	Chemical Process Simulation
10	rdist1	4,134	4,134	0.9945	0.0588	27	65.19 MB	Chemical Process Simulation
11	crystm01*	4,875	4,875	0.9956	1	28	90.66 MB	Materials
12	hydr1	5,308	5,308	0.9992	0.0041	44	0.10 GB	Chemical Process Simulation
13	bayer03	6,747	6,747	0.9994	0.0031	57	0.17 GB	Chemical Process Simulation
14	Lederberg	8,843	8,843	0.9995	0.003	186	0.29 GB	Directed Multigraph
15	circuit_3	12,127	12,127	0.9997	0.7701	140	0.55 GB	Circuit Simulation
16	bayer10	13,436	13,436	0.9996	0.003	158	0.67 GB	Chemical Process Simulation
17	bayer02	13,935	13,935	0.9997	0.002	96	0.72 GB	Chemical Process Simulation
18	crystm02*	13,965	13,965	0.9983	1	82	0.73 GB	Materials
19	bayer04	20,545	20,545	0.9998	0.0016	249	1.57 GB	Chemical Process Simulation
20	crystm03*	24,696	24,696	0.9990	1	239	2.27 GB	Materials

ensuring minimal interference when verifying its superiority over GPU/CPU-based approaches and its advantages over existing CIM-based schemes. (3) NeuroSim provides a fully detailed CIM architecture, which has been widely applied and verified in various studies [14, 52, 57], making it suitable for direct comparisons.

Implementation Details. We implement HyperMR in C++. In the cost model (§3), the bandwidth parameters are $B_0 = 2.27$ GB/sec and $B_1 = 5.26$ GB/sec. The number of Adders are $N_{\text{Adder}} = 64$ and $N'_{\text{Adder}} = 1$. The hardware coefficients are $k_0 = 1.01$ ns, $k_1 = 0.06$ ns, and $k_2 = 1.90$ ns. These parameters are the default settings of NeuroSim that have been validated [39]. Similar to the baselines, the CIM device’s storage capacity is configured to accommodate the largest matrix without exceeding the validated maximum of 1024 tiles [37]. The codebase of HyperMR is made accessible [4].

Benchmarks. On typical MVM operations, we evaluated twenty matrices from the SuiteSparse Matrix Collection [17], as detailed in Table 1, which are widely used in CIM-based storage studies [15, 25, 46]. *Sparsity* indicates the proportion of zero elements in the matrix; all used matrices exhibit high sparsity, presenting greater optimization challenges and highlighting the strengths and weaknesses of different methods. *Symmetry* signifies the extent of matrix symmetry (1 for symmetric, < 1 for asymmetric). The symmetric matrices in Table 1 are marked with stars. Asymmetric matrices, with their complex structures, are more challenging to optimize, as shown in Figure 7. *Used Tiles* represents the number of tiles needed for storage, and *Size* refers to the in-memory size of the matrix. *Application* delineates the real-world applications for the generation of these matrices and their associated MVM operations.

For varied access patterns, we use the top 20 astronomical images from the Sloan Digital Sky Survey (SDSS) archives [54], queried on SkyServer [53] with the statement “SELECT TOP 20 specobjid as name, ra, dec FROM SpecObj”. The following query workloads are considered. (1) **Synthetic Workloads:** For each astronomical image, we generate 25 single-MVM queries, accessing rows and columns of 2D ranges that increase from 0% to 100% at 20% intervals; (2) **Scientific Image Filtering:** This real-world array database workload [9] involves a multi-MVM query using a convolution kernel over the whole image. We set the convolution stride as one and use 12 kernel sizes, such as 3×3 and 5×5 . Both query workloads are used to assess the effect of varied access patterns in §6.4.

Table 2. Comparisons with CIM-based storage schemes. Method “None” means that the matrix is not optimized. The best execution time and CR on each matrix are bolded. *Valid Optimization* is highlighted with a grey background.

Matrix	None	GSMR		TraNNsformer		kMeans		METIS		GMR		ReSpar		HyperMR	
Name	Time (ns)	Time (ns)	CR	Time (ns)	CR	Time (ns)	CR	Time (ns)	CR	Time (ns)	CR	Time (ns)	CR	Time (ns)	CR
qh882	2,943	2,544	1.00	2,737	1.00	2,845	1.00	\	\	3,129	1.00	2,480	1.00	2,497	1.33
radfr1	2,785	2,780	1.00	2,873	1.00	3,485	0.78	\	\	3,481	1.00	3,156	1.00	2,620	1.40
ilic1850	5,106	3,732	1.33	4,348	1.00	4,146	1.00	\	\	3,761	1.00	3,789	1.14	4,109	1.60
qh1484*	5,191	3,878	1.29	4,587	1.00	4,749	1.00	3,921	1.29	5,168	1.00	3,778	1.13	3,813	1.50
rdist3a	5,685	5,560	1.23	6,234	1.07	8,230	0.67	\	\	9,066	0.64	9,054	0.84	5,530	1.23
extr1	7,332	6,451	1.31	7,054	0.88	8,536	0.58	\	\	9,464	0.60	6,788	0.84	6,038	1.75
zenios*	3,231	3,072	3.33	3,190	1.00	3,201	2.50	2,989	1.67	3,186	10.00	3,159	10.00	2,948	1.67
bayer09	7,962	7,459	1.61	7,392	1.26	8,766	0.73	\	\	9,550	0.69	7,848	0.94	6,480	1.81
rdist2	7,042	7,168	1.11	7,789	0.78	11,288	0.51	\	\	12,974	0.48	13,807	0.64	6,787	1.40
rdist1	9,074	9,102	1.08	9,824	0.87	13,431	0.55	\	\	14,071	0.42	20,531	0.51	8,538	1.35
crystm01*	14,129	11,621	1.00	12,243	0.82	22,821	0.31	11,397	0.82	26,855	0.28	13,253	0.64	11,219	1.08
hydr1	13,025	11,869	1.42	12,716	0.96	16,177	0.41	\	\	17,195	0.41	11,588	0.98	10,672	1.76
bayer03	16,574	15,399	1.43	16,219	0.98	21,848	0.38	\	\	23,660	0.34	16,180	0.77	13,625	1.58
Lederberg	22,025	21,271	2.07	24,922	1.50	23,437	0.69	\	\	25,838	1.29	19,634	1.77	16,189	2.00
circuit_3	33,224	41,297	0.77	\	\	41,640	0.34	\	\	43,217	0.37	31,802	0.76	28,006	1.56
bayer10	33,049	33,606	2.00	\	\	44,570	0.31	\	\	47,011	0.24	30,017	0.97	26,550	1.36
bayer02	31,078	30,448	1.17	\	\	47,343	0.22	\	\	51,344	0.16	29,542	0.85	26,772	1.32
crystm02*	47,396	34,791	1.00	\	\	73,661	0.20	34,465	0.53	82,536	0.15	41,384	0.39	31,546	1.04
bayer04	49,794	53,094	1.43	\	\	69,289	0.28	\	\	73,565	0.19	48,792	0.84	40,759	1.05
crystm03*	95,298	63,516	1.65	\	\	136,143	0.27	64,493	0.68	152,556	0.17	82,930	0.57	58,926	1.17
Valid Optimization	0	15		5		4		2		2		5		20	
Best	0	1		0		0		0		0		2		17	

Baselines. We implement 6 competitive CIM-based storage schemes: 1) GSMR [15], based on bandwidth reduction; 2) TraNNsformer [2], based on graph clustering; 3) METIS [20], based on graph partitioning; and 4) kMeans [35], 5) ReSpar [25], and 6) GMR [40], three heuristic ones. Notably, METIS is designed to handle adjacency matrices of graphs and is therefore incapable of optimizing asymmetric matrices, including astronomical images. TraNNsformer was unable to produce results for matrices 15 through 20 due to the memory requirements exceeding the upper limit that TraNNsformer could handle, as indicated in Table 2. Detailed descriptions and comparisons of these baselines have been provided in §7.

Additionally, we compare HyperMR with Apache MADlib [24] and NVIDIA cuSparse [44] in accelerating MVM operations. MADlib is the most commonly used linear algebra module for PostgreSQL and runs on the same CPU server. cuSparse, the prestigious linear algebra library for NVIDIA GPUs, is widely employed in data analysis frameworks such as PyTorch, and we run it on an NVIDIA A100 GPU with 40GB of memory.

Measurement. We use four evaluation metrics: *compression ratio (CR)*, *execution time*, *matrix write time*, and *storage optimization time*. The *CR* is defined as the ratio of tiles used by the unoptimized matrix to those used by the reordered matrix. It should be noted that this *CR* calculation method employed in Table 2 is only applicable to CIM devices. In Table 1, the ‘crystm03’ occupies 2.27 GB of in-memory size in main memory; however, within CIM devices, it utilizes only 239 CIM tiles, corresponding to an actual CIM memory size of 119.5 MB. The *execution time* measures the latency of handling multi-MVM queries specifically in the warm execution phase (i.e., query execution phase in §2.2). The *matrix write time* includes the time needed to transfer the matrix from the main memory to the CIM device and then write it into the device. The *storage optimization time* is the time taken to generate the optimized matrix storage layout. Together, these are referred to as the time overhead (see §6.3), occurring during a cold start scenario. Differentiating between the warm execution phase and the cold start phase enables a more organized evaluation. A storage scheme provides **valid optimization** for a matrix if 1) the *CR* of the optimized matrix is ≥ 1 ; and 2) The execution time after optimization is lower than that of the unoptimized matrix. A storage scheme is considered **the best** for a matrix if 1) the *CR* of the optimized matrix is ≥ 1 ; 2) the execution time after optimization is lower than that of other schemes.

6.2 Efficiency of MVM (RQ1)

6.2.1 HyperMR vs CIM-based Methods. Table 2 presents a detailed comparison of CIM-based baselines in terms of execution time on typical MVM workloads and the CR of the matrices. The matrices listed in Table 1 represent weight matrices of scientific computing models, which are sourced from real scientific computing applications. In warm execution scenarios, these matrices are frequently involved in computations with diverse input vectors. As such, there exists a significant opportunity to optimize their storage layout, thereby accelerating scientific computations.

Compared to the “None” method before storage optimization, HyperMR achieves an average execution time acceleration of 17.32% and an average CR improvement of 45.76%. The efficiency of HyperMR is achieved via optimizations of **O1** and **O2**, which not only speed up execution but also enhance compression by concentrating zero elements in the optimized matrix to maximize idle tiles.

GSMR is a bandwidth reduction-based scheme that assumes a diagonal block pattern exists in matrices. Consequently, GSMR is challenging to optimize all matrices and fails to provide valid optimization for 25% of all evaluated matrices. In contrast, HyperMR offers valid optimization for all matrices without being limited to specific matrix structures. HyperMR outperforms GSMR with an average execution time acceleration of 11.73% and an average CR enhancement of 11.88%. Moreover, HyperMR achieves the fastest execution times in 17 matrices, while GSMR does so in only one.

TraNNsformer is a graph clustering-based scheme limited by its inability to eliminate a few connections between clusters, leading to the allocation of tiles for a few non-zero elements and thus causing hardware resource waste. Furthermore, it does not consider performance optimization. TraNNsformer provides valid optimization for 25% of the matrices. HyperMR outperforms TraNNsformer with an average execution time acceleration of 16.3% and an average CR enhancement of 53.69%.

METIS is a graph partitioning-based scheme that treats the matrix as an adjacency matrix of a graph. Due to its design for special symmetric matrices (i.e., adjacency matrices), METIS cannot handle asymmetric matrices. Moreover, METIS shares a similar limitation with TraNNsformer, leading to hardware resource waste. Thus, METIS only provides valid optimization for 40% of symmetric matrices. HyperMR outperforms METIS with an average execution time acceleration of 4.9% and an average CR enhancement of 43.25%.

kMeans, GMR, and ReSpar are heuristic-based schemes that differ in the metrics they use to swap row and column orders. kMeans employs the k -means algorithm to swap columns, GMR utilizes the density of non-zero entries to swap rows and columns, and ReSpar employs the XOR operation to swap rows. These schemes assume matrix patterns that conform to their heuristic rules, making them less effective for different matrix patterns. HyperMR accelerates average execution time by 55.9%, 68.76%, and 27.74% compared to kMeans, GMR, and ReSpar, respectively. Moreover, the average CR of HyperMR is $3.16 \times$, $3.61 \times$, and $1.69 \times$ that of kMeans, GMR, and ReSpar, respectively. Notably, GMR and ReSpar achieve a high CR (“10.00”) on a special case, “zenios”. Our performance breakdown analysis (cf. Appendix §A.4 [5]) indicates that their heuristic rules consolidate all non-zero entries into a single tile, which caters to the distinctive matrix pattern of “zenios”, i.e., non-zero entries densely concentrated in the upper left corner of the matrix (see our plot of the “zenios” matrix⁵). However, these rules also result in longer tile execution times, and thus, HyperMR is faster than GMR and ReSpar for this special case. In comparison, HyperMR has a maximum tile execution time of 348 ns, compared to 585 ns for GMR and 559 ns for ReSpar. Moreover, kMeans only provides valid optimization for 20% of the matrices in Table 2, GMR for

⁵<https://github.com/HyperMR-project/HyperMR/blob/main/result/zenios.png>

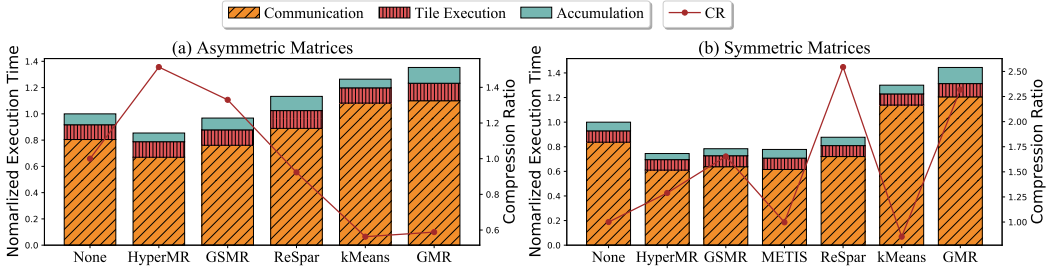


Fig. 7. Results on symmetric and asymmetric matrices.

10%, and ReSpar for 25%. In contrast, HyperMR provides valid optimizations for all matrices. This is aligned with our observation in L1 (see §1) that focusing solely on CRs does not always lead to the efficient acceleration of MVM operations.

It is observed that the CR values achieved by all CIM-based storage schemes generally fall below 2, primarily due to the inherent efficiency of CIM’s original storage mechanism, which does not allocate tiles for zero-filled sub-matrices. As illustrated in Table 2, only 5 out of 119 CR measures surpass 2, constituting a negligible fraction. Notably, four of these exceptional cases originate from a special case “zenios”. Nevertheless, Table 2 shows that HyperMR attains the highest CRs across 75% of cases.

Figure 7 (a) and (b) show the execution time and CRs of all methods on **asymmetric and symmetric matrices**, respectively. The Y-axis labeled ‘Normalized Execution Time’ represents the execution time of each method normalized against the None method. HyperMR primarily optimizes execution time by reducing communication cost, a major performance bottleneck, and outperforms other methods for both symmetric and asymmetric matrices.

In summary, HyperMR provides valid optimization for all evaluated matrices with various matrix structures, a feat that existing research has been unable to achieve. Also, Table 2 demonstrates HyperMR as the best for 17 matrices and competitive with the rest.

6.2.2 HyperMR vs. CIM-based Methods on Denser Matrices. The benchmark matrices used in previous studies (see Table 1) have high sparsity to highlight distinctions between various CIM-based storage schemes, as high sparsity inherently suggests greater optimization potential [15, 25]. It is, however, interesting to know how these CIM-based methods perform over matrices with higher density and more intricate patterns. Thus, we have supplemented 20 additional matrices⁶ from the same matrix repository [17]. The sparsity lower bound for the used benchmark matrices in Table 1 is 0.988. Matrices with a sparsity below 0.466 can be accommodated by a CIM tile, thus obviating the need for optimization. Their information details are reported in Appendix §A.5 [5].

The experimental results indicate that HyperMR provides valid optimization for all denser matrices, achieving the best in 90% of the matrices (see Table 5 in §A.5). In contrast, GMR can only provide valid optimization for 70% of the matrices, TraNNSformer for 20%, kMeans for 30%, METIS for 15%, GMR for 35%, and ReSpar for 25%. The real-world matrices are highly sparse⁷. Also, using widely-used benchmark matrices (Table 1) ensures fairness of comparisons. Thus, the main experimental results in §6 are based on the benchmark matrices, with additional experiments detailed in §A.5 [5].

⁶Additional matrices with sparsity levels ranging from 0.466 to 0.988

⁷Within this large-scale repository, real-world matrices with a sparsity greater than 98.8% account for 78.01%.

6.2.3 HyperMR vs. CPU-based and GPU-based Methods. Figure 8 illustrates the performance benefits of CIM by comparing Apache MADlib [24] on CPU and NVIDIA cuSparse [44] on GPU in processing MVM operations. Both methods are tested in a query execution phase with matrices cached in memory and the devices pre-warmed. Under the *same* warm execution phase we study on (see §2.2), for CPU-based and GPU-based methods, matrix data is read from main memory and GPU memory, respectively, to the CPU and GPU devices. The matrices we evaluated are originally in Coordinate Format (COO) and are sourced from the SuiteSparse Matrix Collection [17]. MADlib and cuSparse can directly perform MVM operations on matrices in this widely-used COO format. The results show that HyperMR is, on average, $251,574 \times$ faster than MADlib and $174 \times$ faster than cuSparse.

In CPU and GPU architectures, computational steps of MVM operations require frequent memory I/O and a significant number of clock cycles for addition and multiplication. In contrast, the CIM architecture demonstrates a substantial performance advantage at the architectural level because data can be computed in place, and addition and multiplication can be completed in a single computational step. Therefore, we found that the latest H100 GPU exhibits a peak arithmetic performance only 2.5 times greater than that of the A100 GPU [32], while our evaluation reveals that CIM boasts a performance advantage more than two orders of magnitude superior to the A100 GPU. Our work focuses on enhancing the CIM storage scheme; thus, we exclude MADlib and cuSparse from subsequent experiments due to their significantly inferior performance.

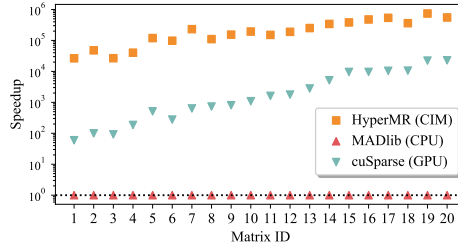


Fig. 8. The speedups: HyperMR (CIM-based) and cuSparse (GPU-based) vs MADlib (CPU-based).

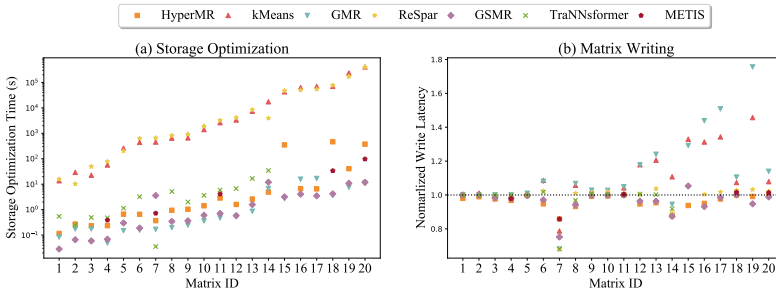


Fig. 9. Time overheads of all schemes.

6.3 Time and Storage Overheads (RQ2)

To expedite the query execution phase shown in Figure 4, all CIM-based storage schemes necessitate a time overhead for optimizing the matrix storage layout and writing the matrix to the CIM device. Moreover, the optimized matrix storage layout results in reordered indexes, which incur storage overhead for all schemes. Therefore, we compare the time and storage overheads of each scheme.

Time Overhead. The time overhead is divided into two phases: storage optimization and matrix writing for all schemes. Figure 9(a) shows the storage optimization times. On average, HyperMR

is $1798 \times$ faster than ReSpar, $2093 \times$ faster than kMeans, $3.30 \times$ faster than TraNnsformer, and $1.07 \times$ faster than METIS; however, HyperMR is $14.97 \times$ and $17.28 \times$ slower than GSMR and GMR, respectively. Although HyperMR exhibits a lower speed than GSMR and GMR, as described in Sections 6.2 and 6.4, HyperMR surpasses both methods regarding query execution acceleration, compression ratio, and adaptability to diverse matrix patterns and access patterns.

Figure 9 (b) shows matrix write times normalized to the unoptimized matrix write time. All schemes have comparable write times except for GMR and kMeans. HyperMR improves matrix write time by 4% on average without additional overhead. Despite HyperMR primarily focusing on accelerating multi-MVM queries and enhancing matrix compression ratios (§2.2), it also slightly optimizes matrix write times across all cases in cold start scenarios.

The performance benefits of CIM devices primarily stem from the non-volatile retention of matrix data within the devices, thereby eliminating the need for data movement during computation. Furthermore, the time overhead associated with storage optimization and matrix writes can be offset by the substantial volume of multi-MVM query requests in practical applications.

Storage Overhead. The storage overhead refers to the storage space required by all storage schemes for storing the reordered index. The reordered index size for all schemes is the same. For an $M \times N$ matrix A , storing the reordered indexes requires $(M + N) \times 4$ bytes. Indeed, the largest *crystm03* matrix (2.27 GB) in Table 1 incurs only 192.94 KB of index storage, which accounts for just 0.008%.

Summary. HyperMR’s time overhead is superior to most existing methods, and HyperMR does not introduce any additional storage overhead compared to all existing methods.

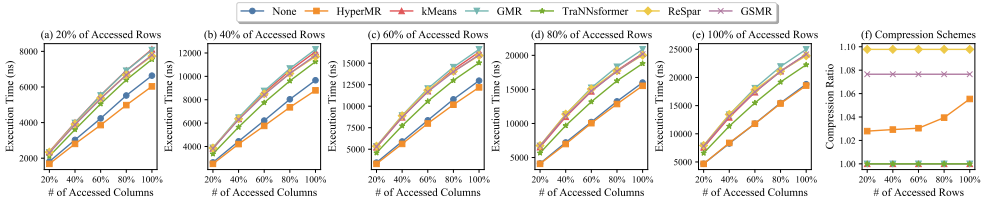


Fig. 10. Comparison of baselines on single-MVM queries in execution time and compression ratios.

6.4 Varied Access Patterns (RQ3)

We compare all evaluated schemes under varied access patterns. Specifically, we set up both single-MVM queries and multi-MVM queries. For single-MVM queries, we adjust the row and column regions accessed to produce different access patterns, gradually increasing from the top 20% of the row and column ranges to the top 100% in 20% increments. For multi-MVM queries, we vary the convolution kernel size to produce different access patterns, ranging from 3×3 to 2401×2401 .

Single-MVM Queries. Figure 10 shows that under varying access patterns, none of the baseline schemes can provide valid optimization, whereas HyperMR consistently does. HyperMR outperforms the baselines in execution time, improving by 29.65% on average, and offers competitive CRs. Figure 10 (a) illustrates a single-MVM query performed on the first 20% of rows starting from row 0 in an image. The x-axis represents the range of columns accessed, increasing from 0% to 100%. Similarly, Figures 10 (b)–(e) follow a similar pattern, with the x-axis indicating the varying parameter (the range of columns accessed). Figure 10 (f) presents the average CRs for Figures 10 (a)–(e). The best CR is 1.1, while HyperMR achieves CRs within the range of 1.02 to 1.06.

Multi-MVM Queries. Figure 11 (a) shows the execution times of all schemes on multi-MVM queries, specifically the Gaussian Smoothing workload, normalized to the unoptimized solution. When the kernel size exceeds 49, the performance differences among all evaluated methods are within 1.6%.

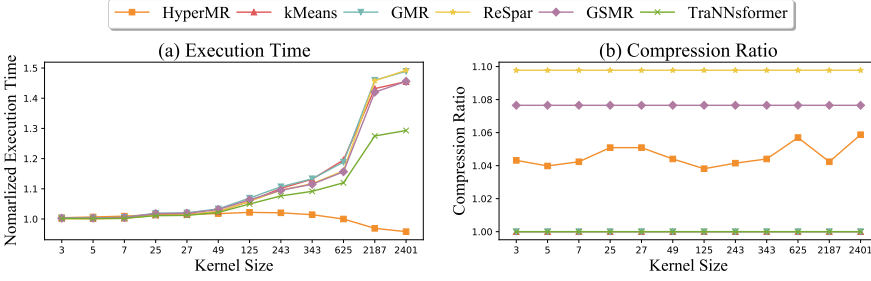


Fig. 11. Comparison of baselines on multi-MVM queries in execution time and compression ratios.

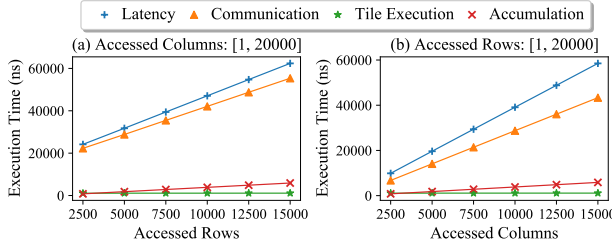


Fig. 12. The lines are the predicted time, and the points are the measured time.

This is because smaller kernel sizes involve fewer tiles in a single MVM operation, leaving less room for optimizing communication and accumulation costs. As the kernel size increases from 125 to 2401, HyperMR exhibits increasing acceleration in execution time compared to the best-performing baseline TraNnsformer, with improvements rising from 2.7% to 34.9%. Figure 11 (b) presents the average CRs. The best CR is 1.1, while HyperMR achieves CRs in the range of 1.03 to 1.06.

Summary. When the access pattern does not uniformly access the entire matrix, existing CIM-based storage schemes generally fail to provide valid optimization. In contrast, HyperMR consistently delivers valid optimization for a wide range of access patterns.

6.5 Cost Model Validation (RQ4)

To validate our cost model proposed in §3, we utilize NeuroSim, which has been previously validated against TSMC’s 40-nm ReRAM-based CIM chip, demonstrating an error rate of under 1% [39]. We examine the model’s ability to estimate execution times for various access patterns. Specifically, we generated 12 single-MVM queries for the largest matrix *crystm03*. These queries fall into two primary types: (1) Fixed column access range of [1, 20000] with varying row access ranges incrementally increasing from [1, 2500] to [1, 15000] in 2500-row intervals; (2) Fixed row access range of [1, 20000] with varying column access ranges incrementally increasing from [1, 2500] to [1, 15000] in 2500-column intervals. The model estimation results indicate that the accuracy of the cost model exceeds 99.9% in all cases. Detailed validation results are presented in Table 7 (see Appendix §A.6 [5]). Figure 12 shows the execution time and the estimated time of the performance components of MVM operations. The predicted values closely align with the actual execution times. We also observe that as the scale of the MVM operation increases, the estimation accuracy for each component’s execution time, as estimated by the cost model, remains consistently high.

Overall, the proposed cost model accurately estimates the execution time of MVM operations. These results indicate that the cost model can provide reliable guidance for designing an efficient storage scheme for the CIM architecture.

7 Related Work

Matrix Storage on CIM Architecture. We classify existing storage schemes into four categories: bandwidth reduction-based [15], graph clustering-based [2, 58], graph partitioning-based [20], and heuristics-based [25, 35, 40]. GSMR [15] is based on the traditional bandwidth reduction algorithm Cuthill-McKee [16]. However, once the diagonal block pattern does not exist in the matrix, the effectiveness of the bandwidth reduction algorithm is significantly decreased [8]. TraNNsformer [2] employs a size-constrained iterative spectral clustering algorithm initially proposed in AutoNCS [58]. It regards the matrix as the connectivity matrix of a bipartite graph, making the connections between nodes within clusters tighter and between clusters sparser. Its limitation is that it tends to produce few connections between clusters but cannot eliminate them, leading to the allocation of storage units for these connections. Similarly, the graph partitioning-based storage schemes [20, 55] treat the matrix as an adjacency matrix of a graph and divide it into disjoint partitions of approximately equal size while minimizing the number of edges connecting different partitions. However, once two partitions have at least one connected edge, they must occupy a storage unit.

The heuristics-based schemes aggregate the non-zero elements based on specific measures. GMR [40] sorts rows and columns based on density. kMeans [35] is the K-means clustering-based column exchanging scheme with the Hamming distance. ReSpar [25] uses the XOR operation to measure between rows. It performs row reordering by computing the common feature and then reorders columns with fewer leading zeros. However, such heuristics need to be more generalizable to diverse matrix patterns. Overall, our experimental evaluation of the existing work concludes that HyperMR, by designing a novel hypergraph-based approach, has achieved significant improvements over existing work.

Optimizing Matrix Storage Layouts. Previous studies [7, 18, 21, 27, 29, 31, 63] have focused on designing efficient linearized storage layouts or compression methods on disks. In contrast, the core contribution of HyperMR is to design efficient matrix storage layouts for newly emerging CIM devices. While HyperMR shares the same motivation as traditional disk-based storage schemes, namely enhancing the MVM-based workloads, its design goals differ due to CIM's unique characteristics; reduced disk I/O and fast decompression are no longer primary concerns. Similar to many efforts exploring novel data storage methods on non-volatile memory devices [3, 30, 45, 56], HyperMR is a novel CIM-based matrix storage method for accelerating multi-MVM queries and improving matrix compression ratios.

Multi-MVM Queries in Data Systems. Many kinds of data systems can benefit from the acceleration efforts for multi-MVM queries, such as: 1) Relational databases, where the OLTP database PostgreSQL supports MVM operations via Apache MADlib [24] and the OLAP database Umbra supports MVM operations [50]; 2) Array databases like SciDB [9], which supports MVM operations and scientific image filtering; 3) In-Database AI [36], where database systems such as Umbra [47] and MonetDB [62] are deeply integrated with AI frameworks (e.g., PyTorch and TensorFlow), leverages NVIDIA cuSparse [44] to support complex deep learning models. HyperMR leverages CIM to accelerate multi-MVM queries and can effectively reduce the latency of data product availability once commodity CIM hardware enters the market.

Matrix Computations on Modern Accelerators. Various devices such as CPU [38], GPU [6], TPU [28], and UPMEM PIM [23] have proven highly effective in accelerating MVM-based workloads. Despite their high performance, these modern accelerators incur significant energy costs when compared to CIM [1, 14, 52, 59], with CIM attaining an efficiency up to 106× than TPU [59]. Recently, GAS [61] proposes an advanced CIM architecture designed to accelerate matrix processing, demonstrating through architectural analysis that the CIM architecture offers superior performance and low energy consumption compared to other architectures. In contrast, HyperMR optimizes

matrix storage layouts on CIM. While GAS cannot serve as a baseline for our work, HyperMR's contributions can synergistically complement those of GAS. In the near future, CIM has the potential to enhance diverse hardware configurations and memory hierarchies in a heterogeneous platform.

8 Conclusion

This study proposes HyperMR, an efficient hypergraph-enhanced matrix storage scheme for the emerging CIM architecture. Compared to existing matrix storage schemes, HyperMR can enable high-quality matrix storage layouts for accelerating MVM-based workloads. It is not limited to specific matrix structures and can leverage various access patterns. Experimental results show that HyperMR outperforms six state-of-the-art storage schemes, offering valid optimization for all evaluated matrices, compared to the best-performing baseline which optimizes only 75%. HyperMR also achieves the best average optimization performance for matrix storage layouts, significantly improving efficiency in varied workload scenarios, with a 29.65% improvement on synthetic queries and up to 34.9% on scientific image filtering.

In future work, we will enhance HyperMR to support three challenging and novel scenarios that existing CIM-based storage schemes have not extensively investigated: 1) more complex workloads, including matrix algebra; 2) matrix computations that exceed the storage capacity of CIM devices; and 3) cold-start scenarios where matrices require frequent updates.

Acknowledgments

We thank the anonymous reviewers and the shepherd for their valuable feedback and suggestions. This research was funded by the Pioneer R&D Program of Zhejiang (Grant No. 2024C01021), the Major Research Program of the Zhejiang Provincial Natural Science Foundation (Grant No. LD24F020015), and Zhejiang Province "Leading Talent of Technological Innovation Program" (No. 2023R5214).

References

- [1] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Faraboschi, Wen-mei W Hwu, John Paul Strachan, Kaushik Roy, and Dejan S. Milojevic. 2019. PUMA: A Programmable Ultra-Efficient Memristor-Based Accelerator for Machine Learning Inference. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 715–731.
- [2] Aayush Ankit, Timur Ibrayev, Abhronil Sengupta, and Kaushik Roy. 2020. TraNNsformer: Clustered Pruning on Crossbar-Based Architectures for Energy-Efficient Neural Networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 10 (2020), 2361–2374.
- [3] Joy Arulraj, Andrew Pavlo, and Subramanya R. Dulloor. 2015. Let's Talk About Storage & Recovery Methods for Non-Volatile Memory Database Systems. In *SIGMOD*. 707–722.
- [4] The HyperMR authors. 2024. The source code of HyperMR, data, and/or other artefacts. <https://github.com/HyperMR-project/HyperMR>. Accessed: 2025-01-09.
- [5] The HyperMR authors. 2024. Supplementary Materials for HyperMR. https://github.com/HyperMR-project/HyperMR/blob/main/doc/HyperMR_Appendix.pdf. Accessed: 2025-01-09.
- [6] Muthu Manikandan Baskaran and Rajesh Bordawekar. 2009. Optimizing sparse matrix-vector multiplication on GPUs. *IBM research report RC24704 W0812-047* (2009), 812–47.
- [7] Sebastian Baunsgaard and Matthias Boehm. 2023. AWARE: Workload-aware, Redundancy-exploiting Linear Algebra. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2023, Seattle, WA, USA, June 18-23, 2023*. ACM, Seattle, WA, USA, 1–1.
- [8] Michael Behrisch, Benjamin Bach, Nathalie Henry Riche, Tobias Schreck, and Jean-Daniel Fekete. 2016. Matrix Reordering Methods for Table and Network Visualization. *Comput. Graph. Forum* 35, 3 (jun 2016), 693–716.
- [9] Paul G. Brown. 2010. Overview of SciDB: large scale array storage, processing and analysis. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*. ACM, Indianapolis, Indiana, USA, 963–968.
- [10] Ümit V. Çatalyürek and Cevdet Aykanat. 1999. Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication. *IEEE Transactions on parallel and distributed systems* 10, 7 (1999), 673–693.

- [11] Ümit V. Çatalyürek, Cevdet Aykanat, and Bora Uçar. 2010. On two-dimensional sparse matrix partitioning: Models, methods, and a recipe. *SIAM Journal on Scientific Computing* 32, 2 (2010), 656–683.
- [12] Indranil Chakraborty, Mustafa Fayeze Ali, Aayush Ankit, Shubham Jain, Sourjya Roy, Shrihari Sridharan, Amogh Agrawal, Anand Raghunathan, and Kaushik Roy. 2020. Resistive Crossbars as Approximate Hardware Building Blocks for Machine Learning: Opportunities and Challenges. *Proc. IEEE* 108, 12 (2020), 2276–2310.
- [13] Pai-Yu Chen, Xiaochen Peng, and Shimeng Yu. 2018. NeuroSim: A Circuit-Level Macro Model for Benchmarking Neuro-Inspired Architectures in Online Learning. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 37, 12 (2018), 3067–3080.
- [14] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, Seoul, Korea, 27–39.
- [15] Jianwei Cui and Qinru Qiu. 2016. Towards memristor based accelerator for sparse matrix vector multiplication. In *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, Montréal, QC, Canada, 121–124.
- [16] E. Cuthill and J. McKee. 1969. Reducing the Bandwidth of Sparse Symmetric Matrices. In *Proceedings of the 1969 24th National Conference (ACM '69)*. Association for Computing Machinery, New York, NY, USA, 157–172.
- [17] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1 (2011), 1:1–1:25.
- [18] Ahmed Elgohary, Matthias Boehm, Peter J Haas, Frederick R Reiss, and Berthold Reinwald. 2016. Compressed linear algebra for large-scale machine learning. *Proceedings of the VLDB Endowment* 9, 12 (2016), 960–971.
- [19] Ben Feinberg, Uday Kumar Reddy Vengalam, Nathan Whitehair, Shibo Wang, and Engin Ipek. 2018. Enabling Scientific Computing on Memristive Accelerators. In *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*. IEEE Computer Society, Los Angeles, CA, USA, 367–382.
- [20] Ben Feinberg, Ryan Wong, T Patrick Xiao, Christopher H Bennett, Jacob N Rohan, Erik G Boman, Matthew J Marinella, Sapan Agarwal, and Engin Ipek. 2021. An analog preconditioner for solving linear systems. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. Seoul, Korea, 761–774.
- [21] Paolo Ferragina, Giovanni Manzini, Travis Gagie, Dominik Köppl, Gonzalo Navarro, Manuel Striani, and Francesco Tosoni. 2022. Improving matrix-vector multiplication via lossless grammar-compressed matrices. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2175–2187.
- [22] Laura Fick, Skylar Skrzyniarz, Malav Parikh, Michael B Henry, and David Fick. 2022. Analog matrix processor for edge AI real-time video analytics. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. San Francisco, CA, USA, 260–262.
- [23] Christina Giannoula, Ivan Fernandez, Juan Gómez Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. 2022. SparseP: Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 1 (2022), 1–49.
- [24] Joseph M. Hellerstein, Christopher Ré, Florian Schoppmann, Daisy Zhe Wang, Eugene Fratkin, Aleksander Gorajek, Kee Siong Ng, Caleb Welton, Xixuan Feng, Kun Li, and Arun Kumar. 2012. The MADlib Analytics Library: Or MAD Skills, the SQL. *Proc. VLDB Endow.* 5, 12 (aug 2012), 1700–1711.
- [25] Yi-Jou Hsiao, Chin-Fu Nien, and Hsiang-Yun Cheng. 2021. ReSpar: Reordering Algorithm for ReRAM-based Sparse Matrix-Vector Multiplication Accelerator. In *39th IEEE International Conference on Computer Design, ICCD 2021, Storrs, CT, USA, October 24-27, 2021*. IEEE, Storrs, CT, USA, 260–268.
- [26] Wei-Hsing Huang, Tai-Hao Wen, Je-Min Hung, Win-San Khwa, Yun-Chen Lo, Chuan-Jia Jhang, Huna-Hsi Hsu, Yu-Hsiana Chin, Yu-Chiao Chen, Chuna-Chuan Lo, Ren-Shuo Liu, Kea-Tiong Tang, Chih-Cheng Hsieh, Yu-Der Chih, Tsung-Yung Chang, and Meng-Fan Chang. 2023. A Nonvolatile AI-Edge Processor with 4MB SLC-MLC Hybrid-Mode ReRAM Compute-in-Memory Macro and 51.4-251TOPS/W. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, San Francisco, CA, 15–17.
- [27] David Johnson, Shankar Krishnan, Jatin Chhugani, Subodh Kumar, and Suresh Venkatasubramanian. 2004. Compressing large boolean matrices using reordering techniques. In *Proceedings of the Thirtieth international conference on Very large data bases-Volume 30*. VLDB Endowment, Toronto, Canada, 13–23.
- [28] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov,

- Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (Toronto, ON, Canada) (ISCA '17). Association for Computing Machinery, New York, NY, USA, 1–12.
- [29] David Kernert, Wolfgang Lehner, and Frank Köhler. 2016. Topology-aware optimization of big sparse matrices and matrix multiplications on main-memory systems. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. Helsinki, Finland, 823–834.
- [30] Hideaki Kimura. 2015. FOEDUS: OLTP Engine for a Thousand Cores and NVRAM. In *SIGMOD*. 691–706.
- [31] Kyoseung Koo, Sohyun Kim, Wonhyeon Kim, Yoojin Choi, Juhee Han, Bogyong Kim, and Bongki Moon. 2024. PreVision: An Out-of-Core Matrix Computation System with Optimal Buffer Replacement. *Proc. ACM Manag. Data* 2, 1, Article 42 (mar 2024), 25 pages.
- [32] Ryota Kusakabe, Kohei Fujita, Tsuyoshi Ichimura, Muneo Hori, and Madgededara Lalith. 2022. GPU-Accelerated Sparse Matrix Vector Product based on Element-by-Element Method for Unstructured FEM using OpenACC. In *SC*. 52–61.
- [33] Manuel Le Gallo, Riduan Khaddam-Aljameh, Milos Stanisavljevic, Athanasios Vasilopoulos, Benedikt Kersting, Martino Dazzi, Geethan Karunaratne, Matthias Brändli, Abhairaj Singh, Silvia M Mueller, et al. 2023. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nature Electronics* 680–693, 6 (2023), 1–14.
- [34] Can Li, Miao Hu, Yunning Li, Hao Jiang, Ning Ge, Eric Montgomery, Jiaming Zhang, Wenhao Song, Noraica Dávila, Catherine E Graves, et al. 2018. Analogue signal and image processing with large memristor crossbars. *Nature electronics* 1, 1 (2018), 52–59.
- [35] Jilan Lin, Zhenhua Zhu, Yu Wang, and Yuan Xie. 2019. Learning the sparsity for ReRAM: Mapping and pruning sparse neural network for ReRAM based accelerator. In *Proceedings of the 24th Asia and South Pacific Design Automation Conference*. Association for Computing Machinery, New York, NY, USA, 639–644.
- [36] Qiuru Lin, Sai Wu, Junbo Zhao, Jian Dai, Feifei Li, and Gang Chen. 2022. A Comparative Study of in-Database Inference Approaches. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 1794–1807.
- [37] Haikun Liu, Jiahong Xu, Xiaofei Liao, Hai Jin, Yu Zhang, and Fubing Mao. 2022. A Simulation Framework for Memristor-Based Heterogeneous Computing Architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 12 (2022), 5476–5488.
- [38] Xing Liu, Mikhail Smelyanskiy, Edmond Chow, and Pradeep Dubey. 2013. Efficient Sparse Matrix-Vector Multiplication on X86-Based Many-Core Processors. In *Proceedings of the 27th International ACM Conference on International Conference on Supercomputing* (Eugene, Oregon, USA) (ICS '13). Association for Computing Machinery, New York, NY, USA, 273–282.
- [39] Anni Lu, Xiaochen Peng, Wantong Li, Hongwu Jiang, and Shimeng Yu. 2021. NeuroSim simulator for compute-in-memory hardware accelerator: Validation and benchmark. *Frontiers in artificial intelligence* 4 (2021), 659060.
- [40] Darya Mikhailenko, Yujin Nakamoto, Ben Feinberg, and Engin Ipek. 2020. Adapting In Situ Accelerators for Sparsity with Granular Matrix Reordering. *IEEE Computer Architecture Letters* 19, 2 (2020), 143–146.
- [41] Mythic. [n. d.]. News. <https://mythic.ai/whats-new/mythic-raises-13-million-to-bring-its-next-generation-analog-computing-solution-to-market/>. Accessed: 2025-01-09.
- [42] Mythic. 2024. M1076 Analog Matrix Processor. <https://mythic.ai/products/m1076-analog-matrix-processor/>. Accessed: 2025-01-09.
- [43] Mythic. 2024. Mythic MP10304. <https://mythic.ai/wp-content/uploads/2022/03/MP10304-Card-Product-Brief-v20211115-16.pdf>. Accessed: 2025-01-09.
- [44] NVIDIA. 2024. cuSPARSE. <https://docs.nvidia.com/cuda/cusparse/index.html>. Accessed: 2025-01-09.
- [45] Ismail Oukid and Wolfgang Lehner. 2017. Data Structure Engineering For Byte-Addressable Non-Volatile Memory. In *SIGMOD*. 1759–1764.
- [46] John K Reid and Jennifer A Scott. 2006. Reducing the total bandwidth of a sparse unsymmetric matrix. *SIAM J. Matrix Anal. Appl.* 28, 3 (2006), 805–821.
- [47] Maximilian Rieger, Moritz Sichert, and Thomas Neumann. 2022. Integrating deep learning frameworks into main-memory databases. In *Proceedings of the VLDB 2022 Applied AI for Database Systems and Applications Workshop co-located with (VLDB 2022)(AIDB Workshop Proceedings)*.
- [48] Sebastian Schlag. 2020. *High-Quality Hypergraph Partitioning*. Ph.D. Dissertation. Karlsruhe Institute of Technology, Germany.
- [49] Sebastian Schlag, Tobias Heuer, Lars Gottesbüren, Yaroslav Akhremtsev, Christian Schulz, and Peter Sanders. 2022. High-Quality Hypergraph Partitioning. *ACM J. Exp. Algorithmics* 27 (2022), 1.9:1–1.9:39.
- [50] Maximilian Schüle, Tobias Götz, Alfons Kemper, and Thomas Neumann. 2021. ArrayQL for Linear Algebra within Umbra. In *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management* (Tampa,

- FL, USA) (*SSDBM '21*). Association for Computing Machinery, New York, NY, USA, 193–196.
- [51] Abu Sebastian, Manuel Le Gallo, Riduan Khaddam-Aljameh, and Evangelos Eleftheriou. 2020. Memory devices and applications for in-memory computing. *Nature nanotechnology* 15, 7 (2020), 529–544.
 - [52] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, Seoul, Korea, 14–26.
 - [53] Alexander S. Szalay, Jim Gray, Ani Thakar, Peter Z. Kunszt, Tanu Malik, M. Jordan Raddick, Christopher Stoughton, and Jan vandenBerg. 2002. The SDSS skyserver: public access to the sloan digital sky server data. In *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data, Madison, Wisconsin, USA, June 3-6, 2002*. ACM, Madison, Wisconsin, USA, 570–581.
 - [54] Alexander S. Szalay, Peter Z. Kunszt, Ani Thakar, Jim Gray, Donald R. Slutz, and Robert J. Brunner. 2000. Designing and Mining Multi-Terabyte Astronomy Archives: The Sloan Digital Sky Survey. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, May 16-18, 2000, Dallas, Texas, USA*. ACM, New York, NY, USA, 451–462.
 - [55] Twisha Titirsha, Shihao Song, Anup Das, Jeffrey L. Krichmar, Nikil D. Dutt, Nagarajan Kandasamy, and Francky Cathoor. 2021. Endurance-Aware Mapping of Spiking Neural Networks to Neuromorphic Hardware. *IEEE Transactions on Parallel and Distributed Systems* 33 (2021), 288–301.
 - [56] Alexander van Renen, Viktor Leis, Alfons Kemper, Thomas Neumann, Takushi Hashida, Kazuichi Oe, Yoshiyasu Doi, Lilian Harada, and Mitsuru Sato. 2018. Managing Non-Volatile Memory in Database Systems. In *SIGMOD*. 1541–1555.
 - [57] Fang Wang, Man Lung Yiu, and Zili Shao. 2021. Accelerating Similarity-based Mining Tasks on High-dimensional Data by Processing-in-memory. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, Chania, Greece, 1859–1864.
 - [58] Wei Wen, Chi-Ruo Wu, Xiaofang Hu, Beiye Liu, Tsung-Yi Ho, Xin Li, and Yiran Chen. 2015. An EDA Framework for Large Scale Hybrid Neuromorphic Computing Systems. In *Proceedings of the 52nd Annual Design Automation Conference (San Francisco, California) (DAC '15)*. Association for Computing Machinery, New York, NY, USA, Article 12, 6 pages.
 - [59] Geng Yuan, Payman Behnam, Zhengang Li, Ali Shafiee, Sheng Lin, Xiaolong Ma, Hang Liu, Xuehai Qian, Mahdi Nazm Bojnordi, Yanzhi Wang, and Caiwen Ding. 2021. FORMS: Fine-grained Polarized ReRAM-based In-situ Computation for Mixed-signal DNN Accelerator. In *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, Valencia, Spain, 265–278.
 - [60] Baogang Zhang, Necati Uysal, and Rickard Ewetz. 2021. Computational Restructuring: Rethinking Image Compression Using Resistive Crossbar Arrays. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 5 (2021), 836–849.
 - [61] Xiaoyu Zhang, Zerun Li, Rui Liu, Xiaoming Chen, and Yinhe Han. 2024. GAS: General-Purpose In-Memory-Computing Accelerator for Sparse Matrix Multiplication. *IEEE Trans. Comput.* 73, 6 (2024), 1427–1441.
 - [62] Ying Zhang, Richard Koopmanschap, and Martin Kersten. 2018. Love at First Sight: MonetDB/TensorFlow. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. 1672–1672.
 - [63] Yi Zhang, Kamesh Munagala, and Jun Yang. 2011. Storing Matrices on Disk: Theory and Practice Revisited. *Proc. VLDB Endow.* 4, 11 (aug 2011), 1075–1086.

Received July 2024; revised September 2024; accepted November 2024